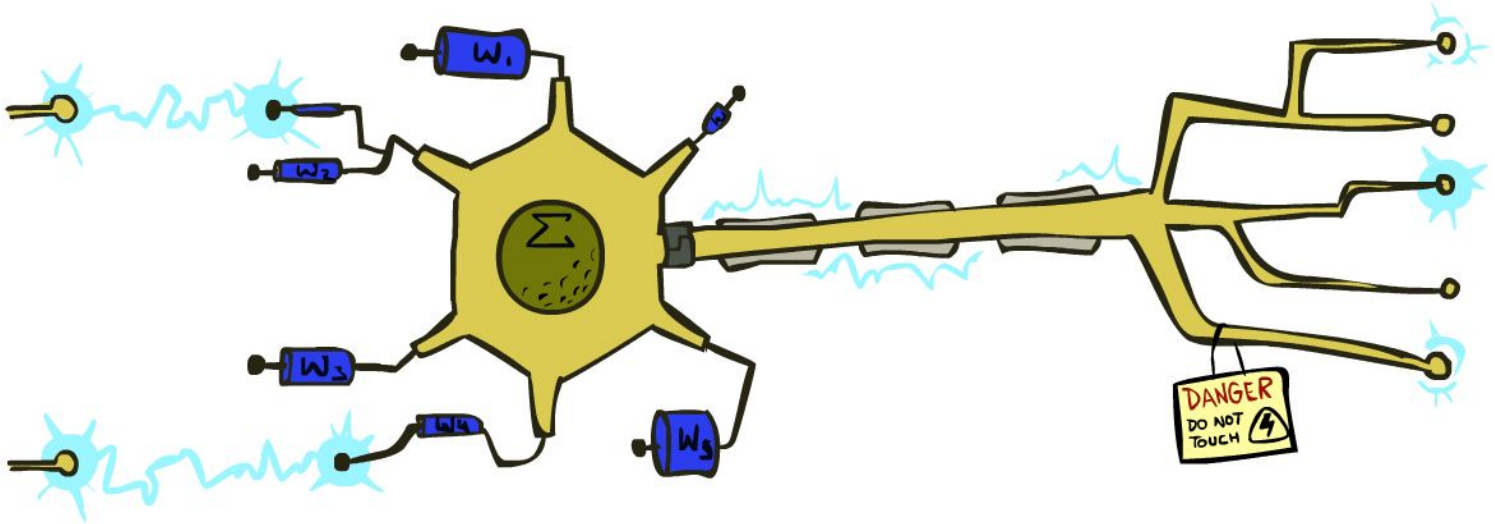
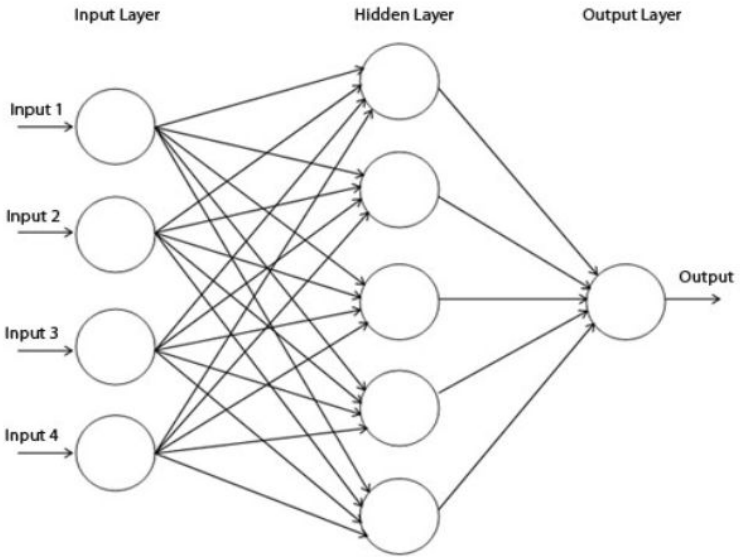
CSE 445: Machine Learning

Neural Networks



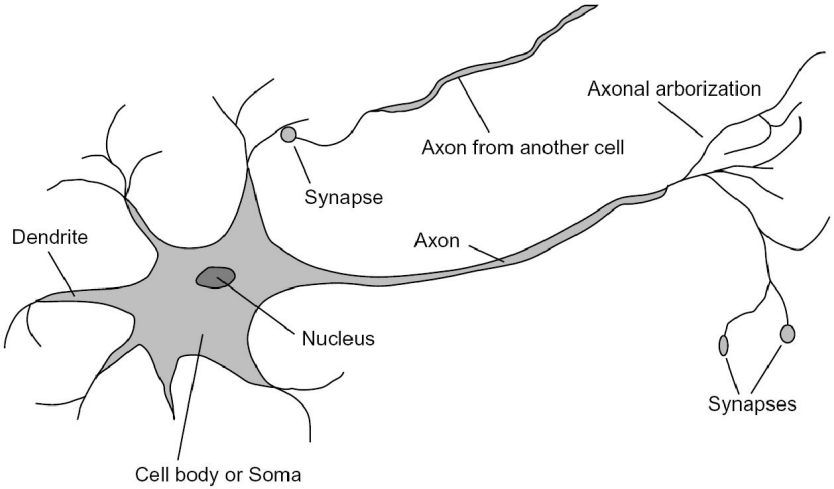
Neural Networks - Motivation

- Neural Networks (NN) are biologically inspired – attempt to build computational models that operate like human brain
- Makes no assumptions about the data – can be very accurate
- Can be used in classification as well as regression
- Black box model



Biological Neurons

- Receive multiple inputs from other cells via the dendrites
- Fires an impulse output via the axon when a threshold has been met
- Arranged in complex layers



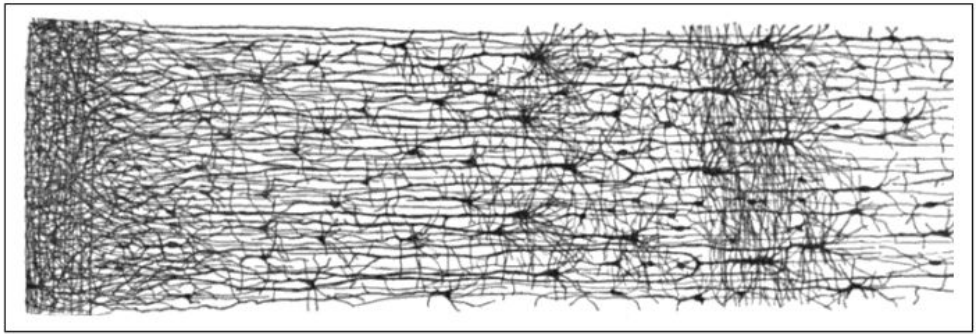
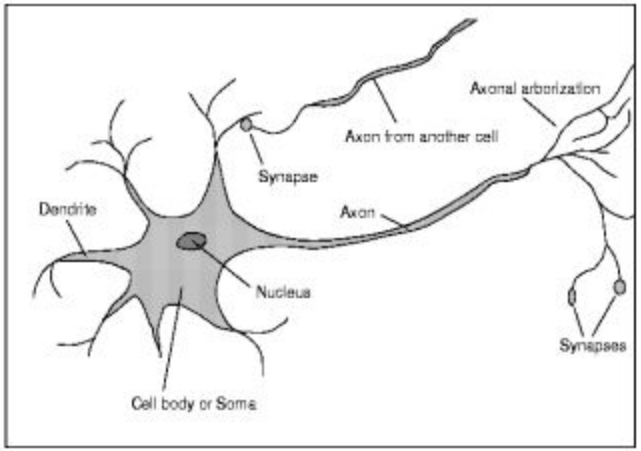
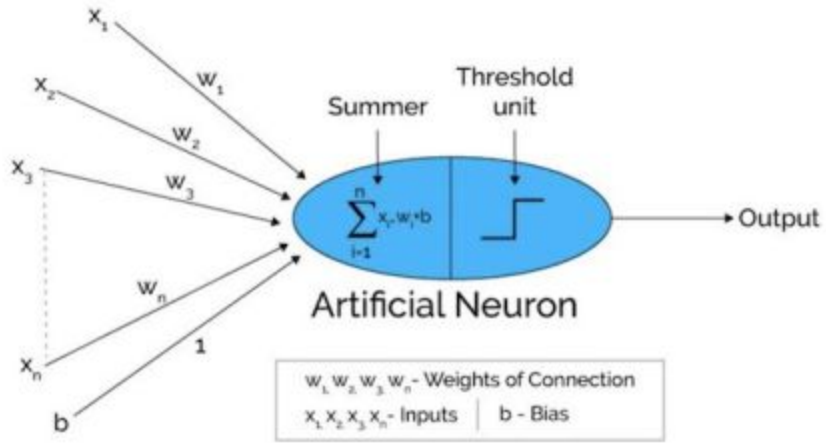


Figure 10-2. Multiple layers in a biological neural network (human cortex)[6]

Artificial Neuron - Perceptron

- Artificial neuron fires when the combined input exceeds threshold
- Inputs and weights can be any value
- Weights are learned
- By itself, cannot solve the XOR problem – solution is to stack multiple layers, and use nonlinear activation functions





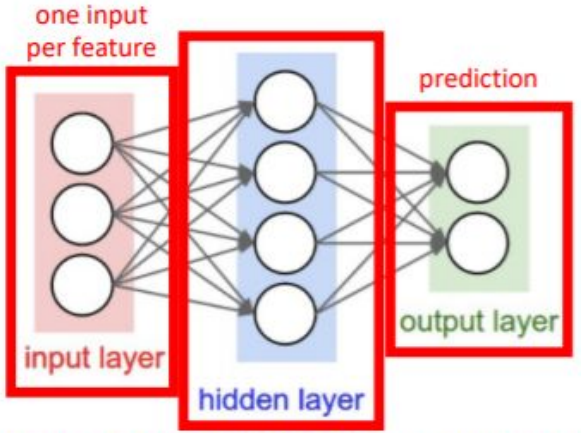
Perceptron math

$$\hat{y} = \Theta(w_1 x_1 + w_2 x_2 + ... + w_n x_n + b)$$
$$= \Theta(\mathbf{w} \cdot \mathbf{x} + b)$$

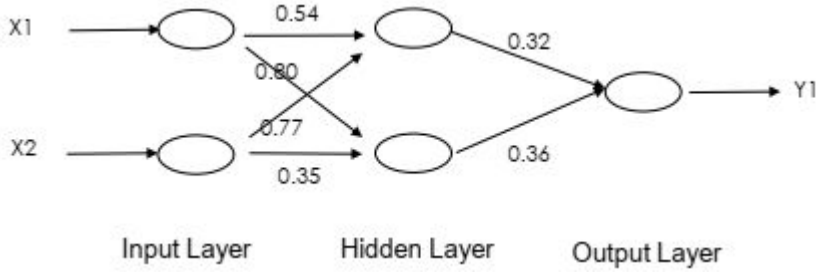
$$\text{where} \quad \Theta(v) = \begin{cases} 1 & \text{if } v \geqslant 0 \\ 0 & \text{otherwise} \end{cases}$$

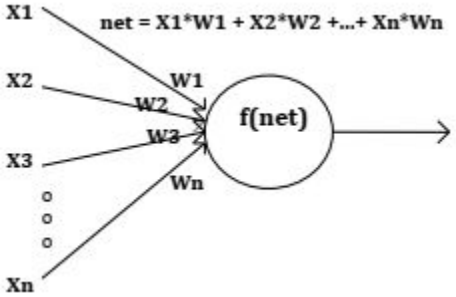
General Architecture – Neural Networks

- Framework (in general, but not for all NNs)
  - Input Layer + Hidden Layer(s) + Output Layer
  - Weights
  - Activation Functions f(net)
- Weights initialized to small real numbers
- Learning rule determine how to update connection weights during each training cycle
- Weight adjustment based on errors/distance



"hidden layer" uses outputs of units (i.e., neurons) and provides them as inputs to other units (i.e., neurons)





Activation Functions

- Activation functions are nonlinear in nature, because:
  - Not possible to use backpropagation to train the model if linear
  - No matter how many layers, the last layer will still be a linear function of the first layer
- Functions with a computationally inexpensive derivative (like ReLU) converge quickly, and are widely used
- Important to avoid vanishing/exploding gradients problem!

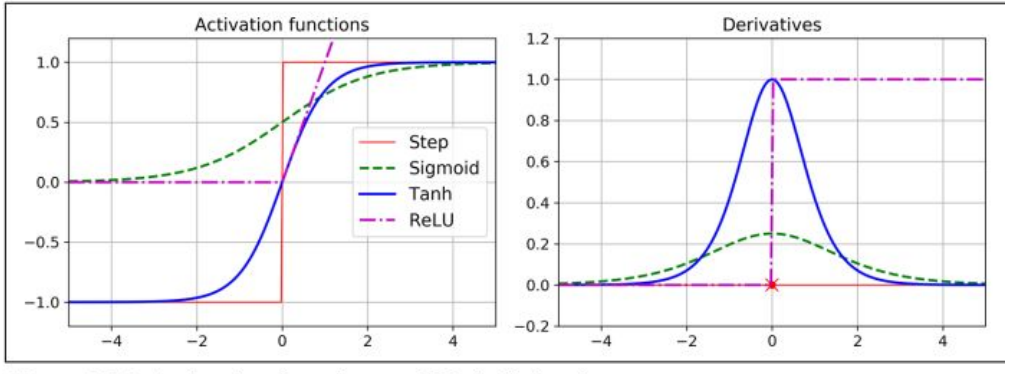
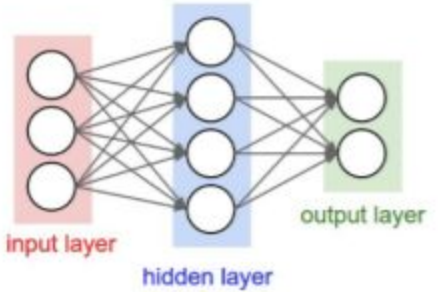


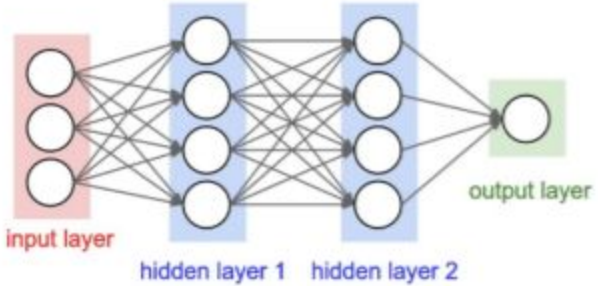
Figure 10-8. Activation functions and their derivatives

ACTIVATION FUNCTION	EQUATION	RANGE
Linear Function	$f(x) = x$	$(-\infty, \infty)$
Step Function	$f(x) = \begin{cases} 0 \; for \; x < 0 \\ 1 \; for \; x \geq 0 \end{cases}$	$\{0, 1\}$
Sigmoid Function	$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$	$(0, 1)$
Hyperbolic Tanjant Function	$f(x) = \tanh(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$	$(-1, 1)$
ReLU	$f(x) = \begin{cases} 0 \; for \; x < 0 \\ x \; for \; x \geq 0 \end{cases}$	$[0, \infty)$
Leaky ReLU	$f(x) = \begin{cases} 0.01 \; for \; x < 0 \\ x \; for \; x \geq 0 \end{cases}$	$(-\infty, \infty)$
Swish Function	$f(x) = 2x\sigma(\beta x) = \begin{cases} \beta = 0 \; for \; f(x) = x \\ \beta \to \infty \; for \; f(x) = 2\max(0, x) \end{cases}$	$(-\infty, \infty)$

Fully Connected, Feedforward Neural Networks

- Fully Connected □ each unit in the network provides an input to each unit in the next layer
- Feedforward □ Each layer serves as input to the next layer, with no loops
- If NN contains a deep stack of hidden layers □ Deep Neural Network
- How to train the network?
  - Answer: Backpropagation algorithm
  - Computes gradient of the network's error with respect to every model parameter efficiently
  - Once the gradients are obtained, just a regular Gradient Descent step used to update params



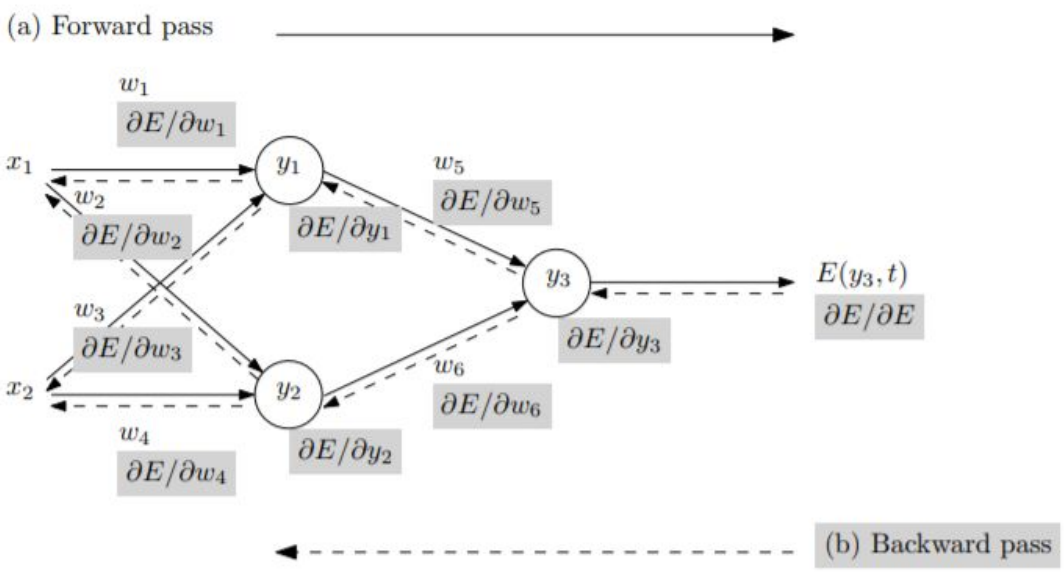


Backpropagation

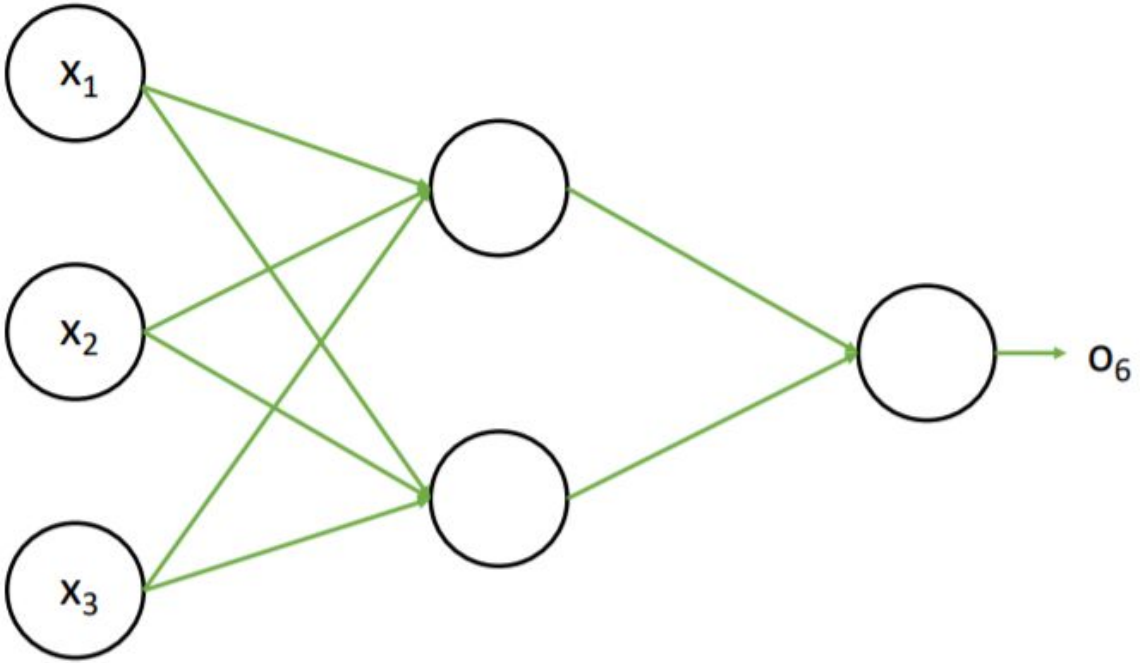
1. Initialize the weights to small random numbers
2. Randomly select a input and output pair (x,y) and present the input to the network. Compute the corresponding predicted output generated by the network. This is a FORWARD PASS.
3. Compute the error between the output generated by the network, and the true labels.
4. Propagate the “blame” for prediction errors back throughout the model, computing the error gradients with respect to each weight (thanks to chain rule!). This is the BACKWARD PASS
5. Update each weight using the gradients computed in #4 using Gradient Descent (recall Lecture #4)
6. If the stopping criterion is met (number of epochs, cost function threshold etc), you’re done! Otherwise repeat 2-5.

Backpropagation

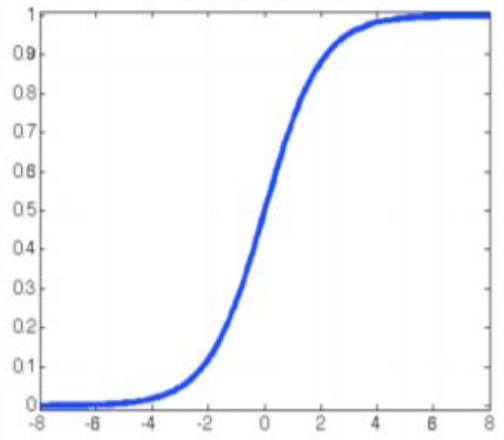
- Forward Pass:
  - Training inputs x_i are fed forward to generate corresponding activations y_i.
  - Error E between generated output y_3 and target output t is computed
- Backward Pass:
  - Error is propagated backwards, giving the gradient with respect to the weights $\nabla_{w_i} E = (\frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_6})$
  - Use a variant of Gradient Descent (batch, stochastic, or mini-batch) to update weights



Backpropagation Example – NN Architecture



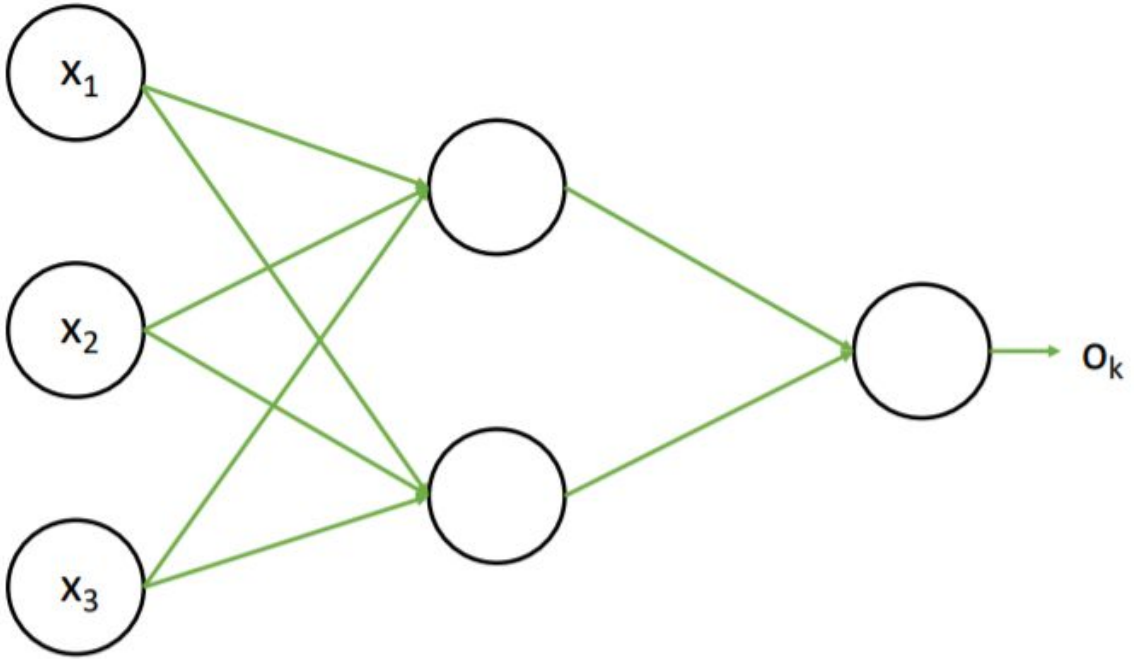
Sigmoid Activation Function



$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

Example from: Jiawei Han and Micheline Kamber; Data Mining.

Backpropagation Example – Loss Function

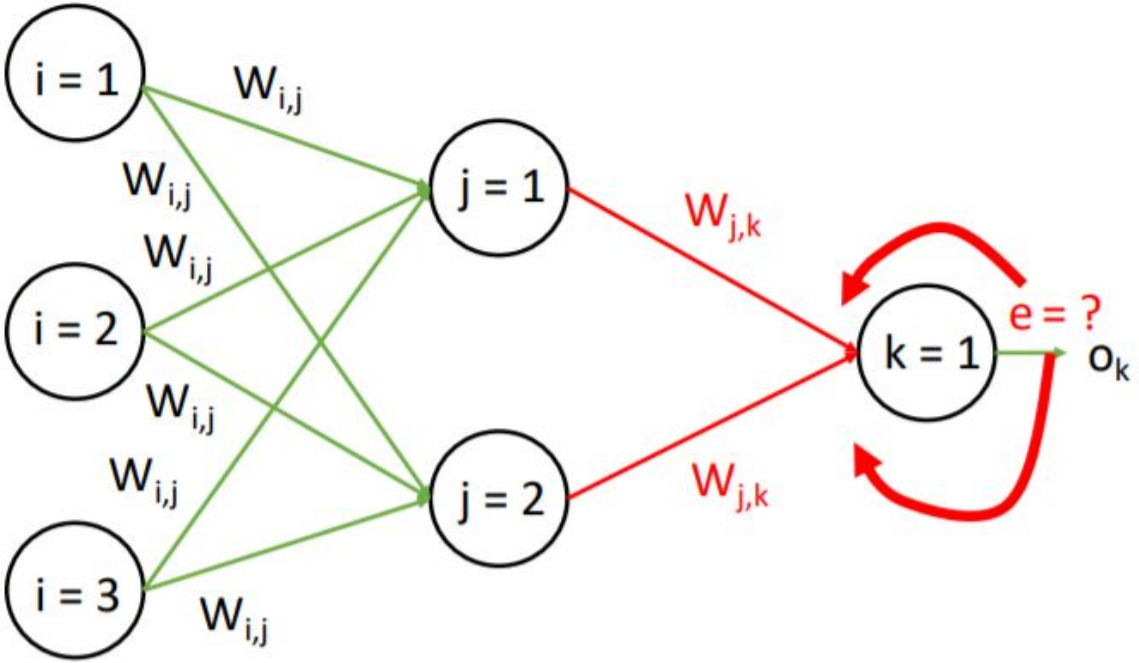


Squared Error Function

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}}(t_k - o_k)^2$$

Example from: Jiawei Han and Micheline Kamber; Data Mining.

Backpropagation Example – Computing Gradient



t is a constant value:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}}(t_k - o_k)^2$$

Example from: Jiawei Han and Micheline Kamber; Data Mining.

Backpropagation Example – Computing Gradient

t is a constant value:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}}(t_k - o_k)^2$$

Using the following chain rule :

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial o_k} * \frac{\partial o_k}{\partial w_{jk}}$$

$$\frac{\partial E}{\partial o_k} = -2(t_k - o_k)$$

Sigmoid activation function: $\sigma(x) = \frac{1}{1+e^{-x}}$

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2}$$

$$\frac{d\sigma(x)}{dx} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right)$$

$$\frac{d\sigma(x)}{dx} = (1-\sigma(x))\,\sigma(x)$$

Backpropagation Example – Output layer gradient

t is a constant value:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}}(t_k - o_k)^2$$

$$\frac{\partial E}{\partial o_k} = -2(t_k - o_k)$$

Sigmoid activation function: $\sigma(x) = \frac{1}{1+e^{-x}}$

Using the following chain rule :

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial o_k} * \frac{\partial o_k}{\partial w_{jk}}$$

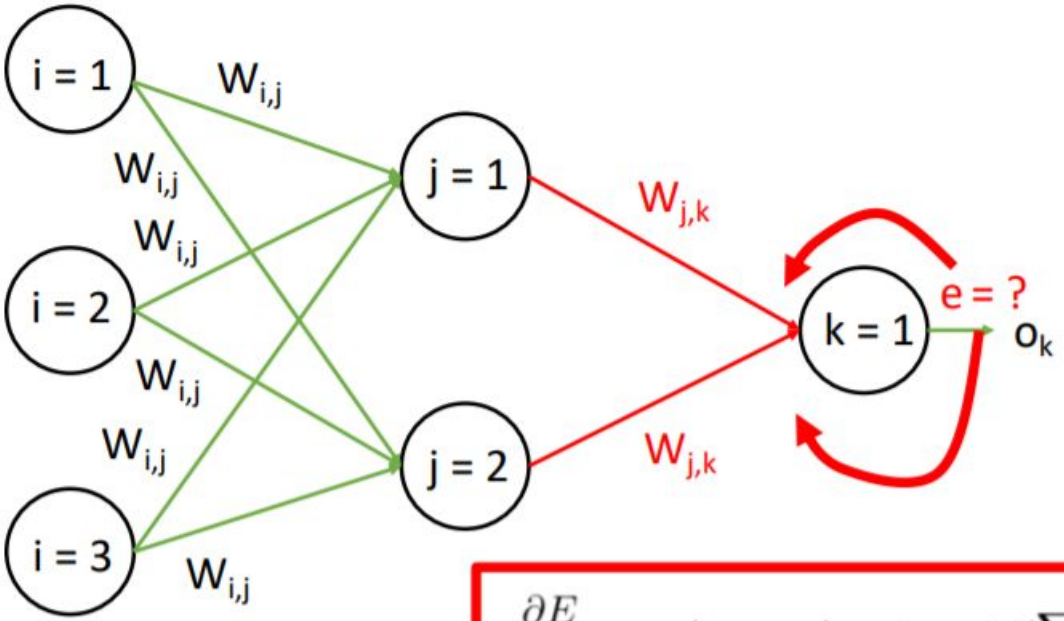
$$\frac{d\sigma(x)}{dx} = (1-\sigma(x))\,\sigma(x)$$

We can rewrite our function as follows:

For efficiency, compute last

$$\frac{\partial E}{\partial w_{jk}} = -2(t_k - o_k)\; sigmoid(\sum_j w_{jk} * o_j) * (1 - sigmoid(\sum_j w_{jk} * o_j)) * o_j$$

Backpropagation Example – Output Layer

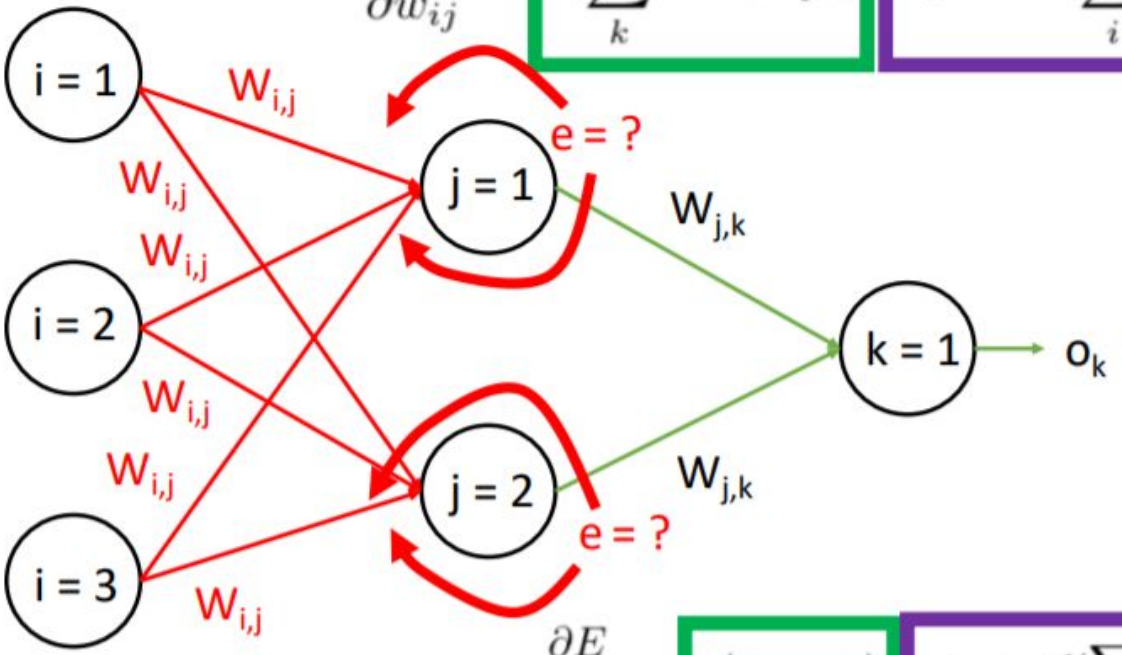


$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial}{\partial w_{jk}}(t_k - o_k)^2$$

$$\frac{\partial E}{\partial w_{jk}} = -(t_k - o_k) * sigmoid(\sum_j w_{jk} * o_j) * (1 - sigmoid(\sum_j w_{jk} * o_j))$$

Backpropagation Example – Hidden Layer

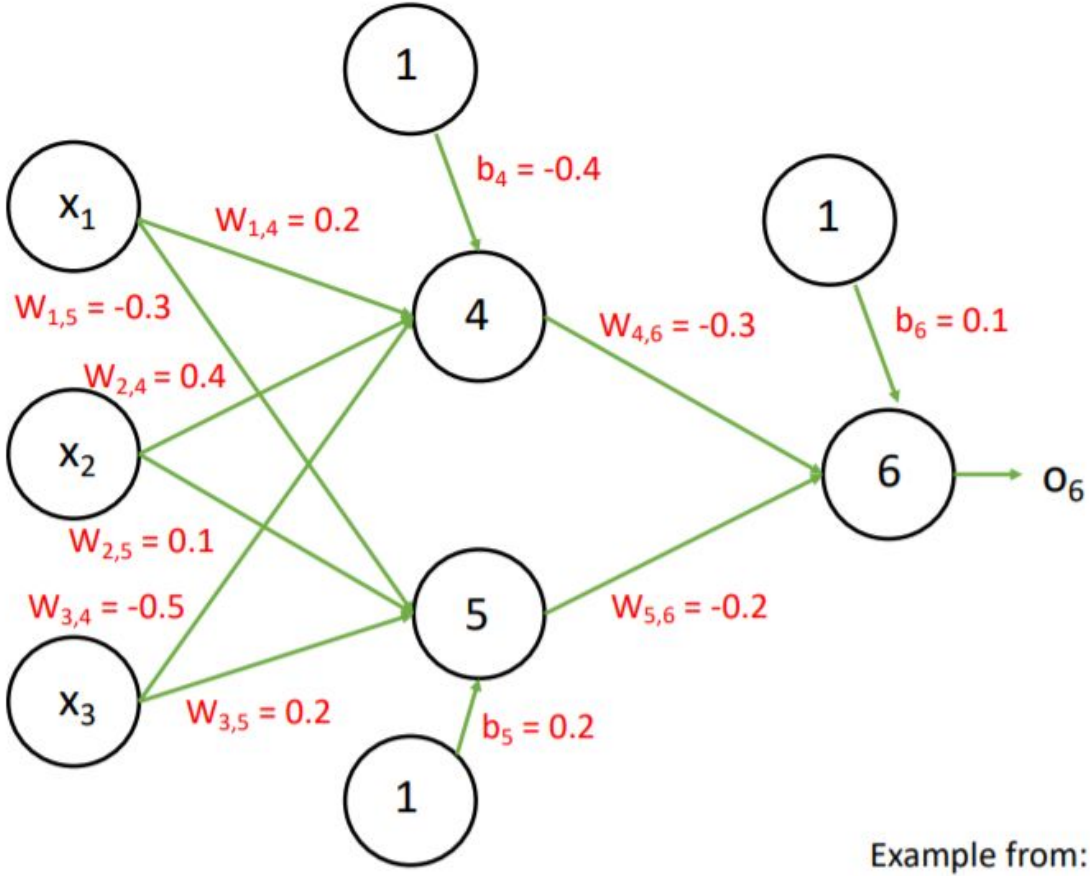
$$\frac{\partial E}{\partial w_{ij}} = -(\sum_k Err_k w_{jk}) \; sigmoid(\sum_i w_{ij} * o_i) * (1 - sigmoid(\sum_i w_{ij} * o_i))$$



Symmetry in how to solve for parameters in hidden layers

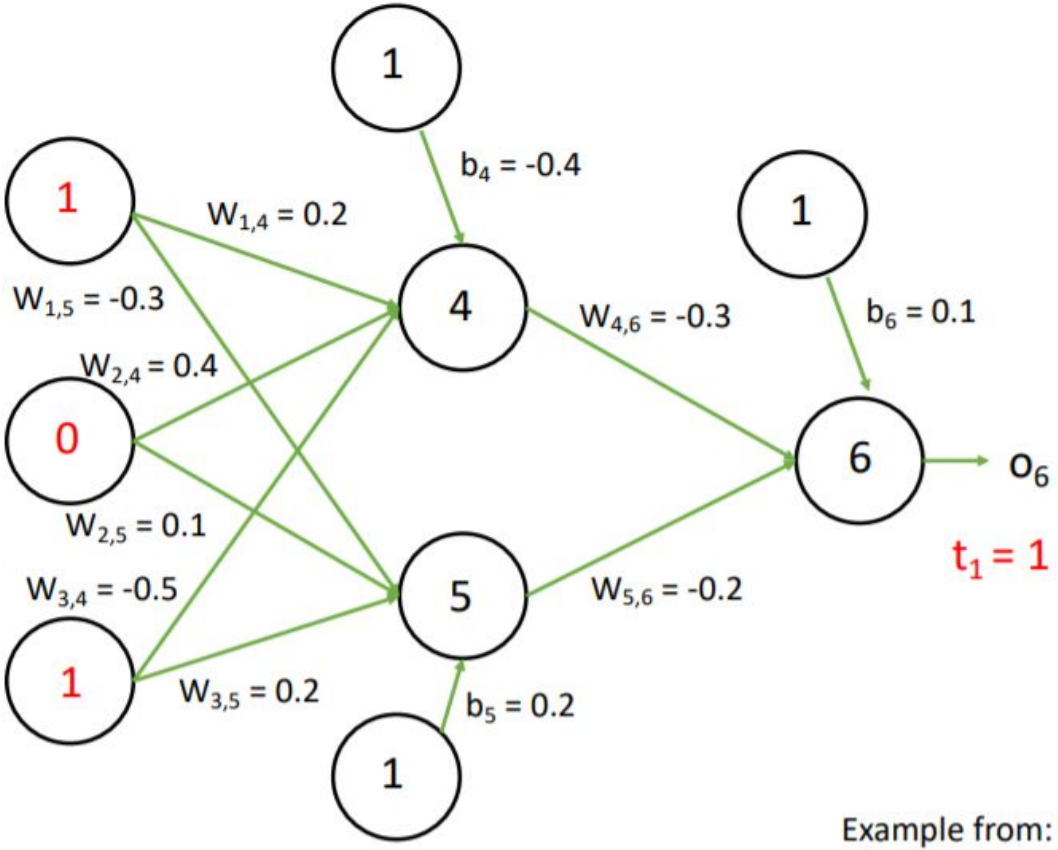
$$\frac{\partial E}{\partial w_{jk}} = -(t_k - o_k) \; sigmoid(\sum_j w_{jk} * o_j) * (1 - sigmoid(\sum_j w_{jk} * o_j))$$

Backpropagation Example – Initial Values



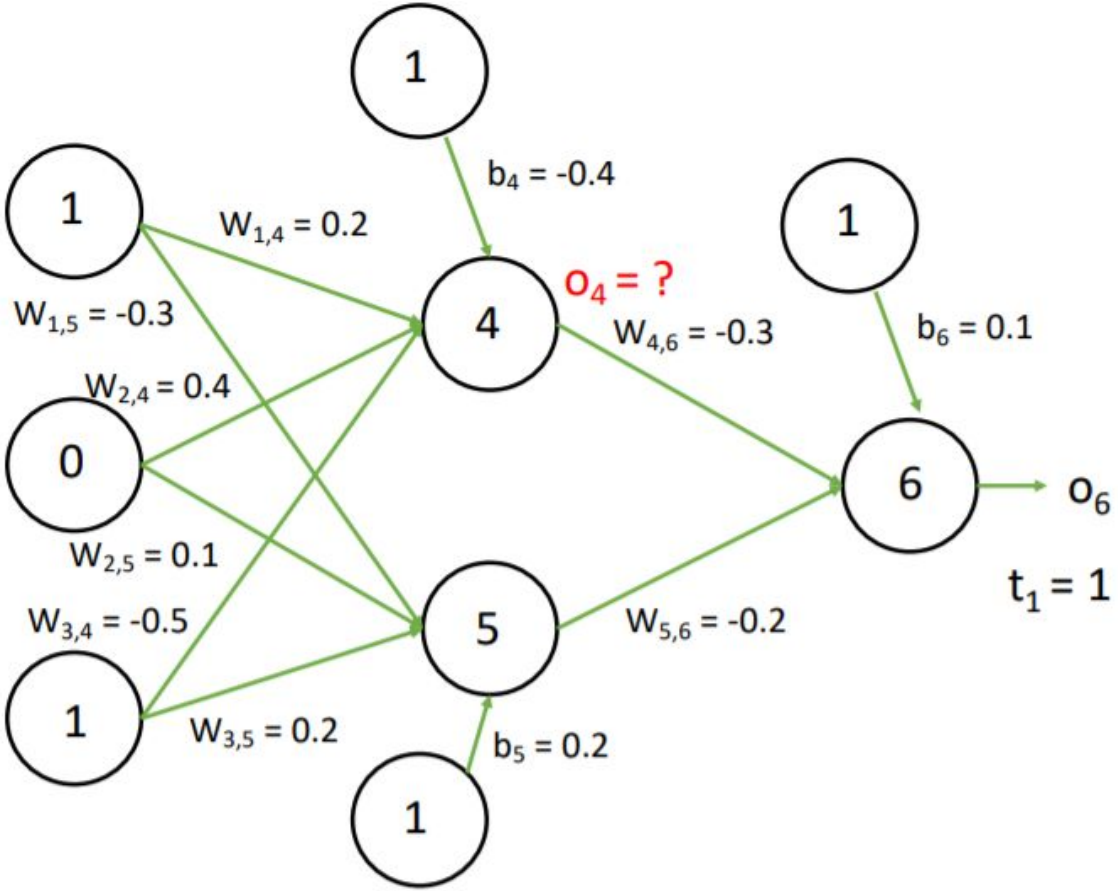
Example from: Jiawei Han and Micheline Kamber; Data Mining.

Backpropagation Example – Input Training example



Example from: Jiawei Han and Micheline Kamber; Data Mining.

Backpropagation Example – Forward Pass



Input to node 4:

$i_4 = (1 \times 0.2 + 0 \times 0.4 + 1 \times -0.5) - 0.4$

$i_4 = -0.7$

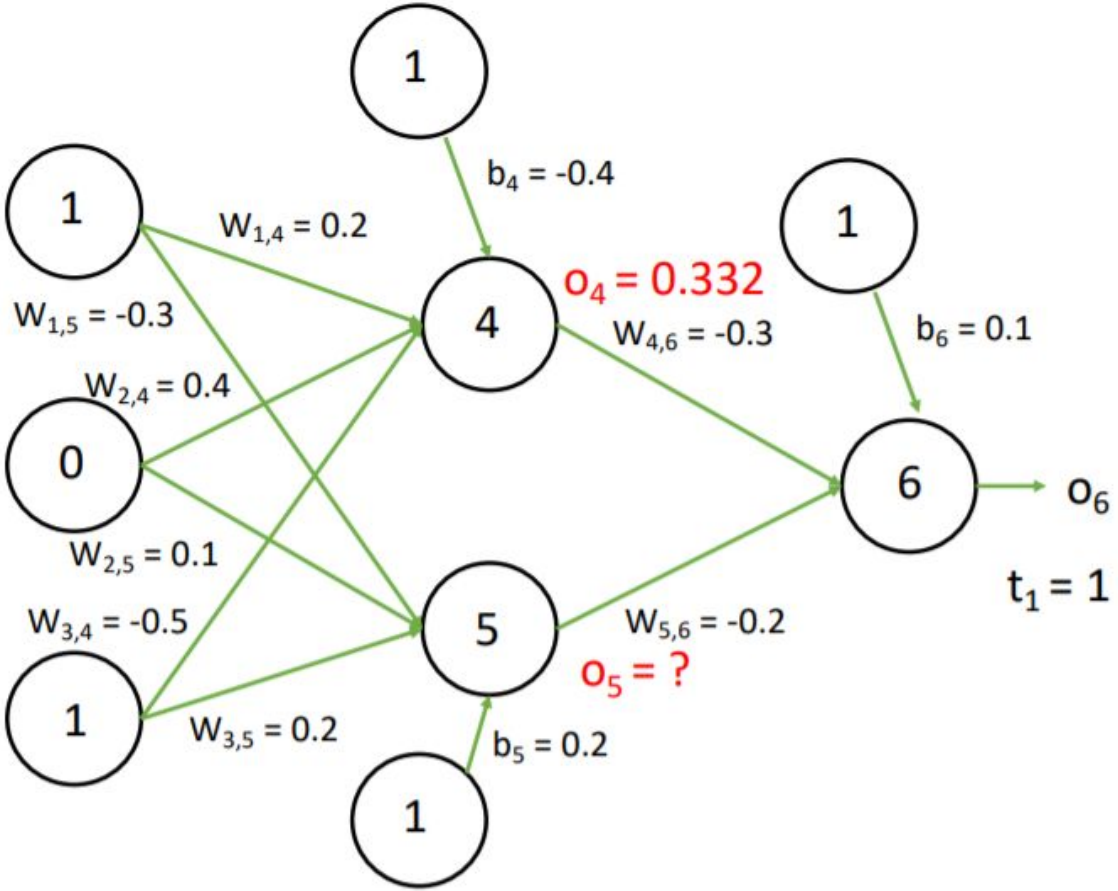
Output of node 4 (sigmoid function):

$o_4 = sigmoid(-0.7)$

$o_4 = 1/(1+e^{-(-0.7)})$

$o_4 = 0.332$

Backpropagation Example – Forward Pass



Input to node 5:

$i_5 = (1 \times -0.3 + 0 \times 0.1 + 1 \times 0.2) + 0.2$

$i_5 = 0.1$

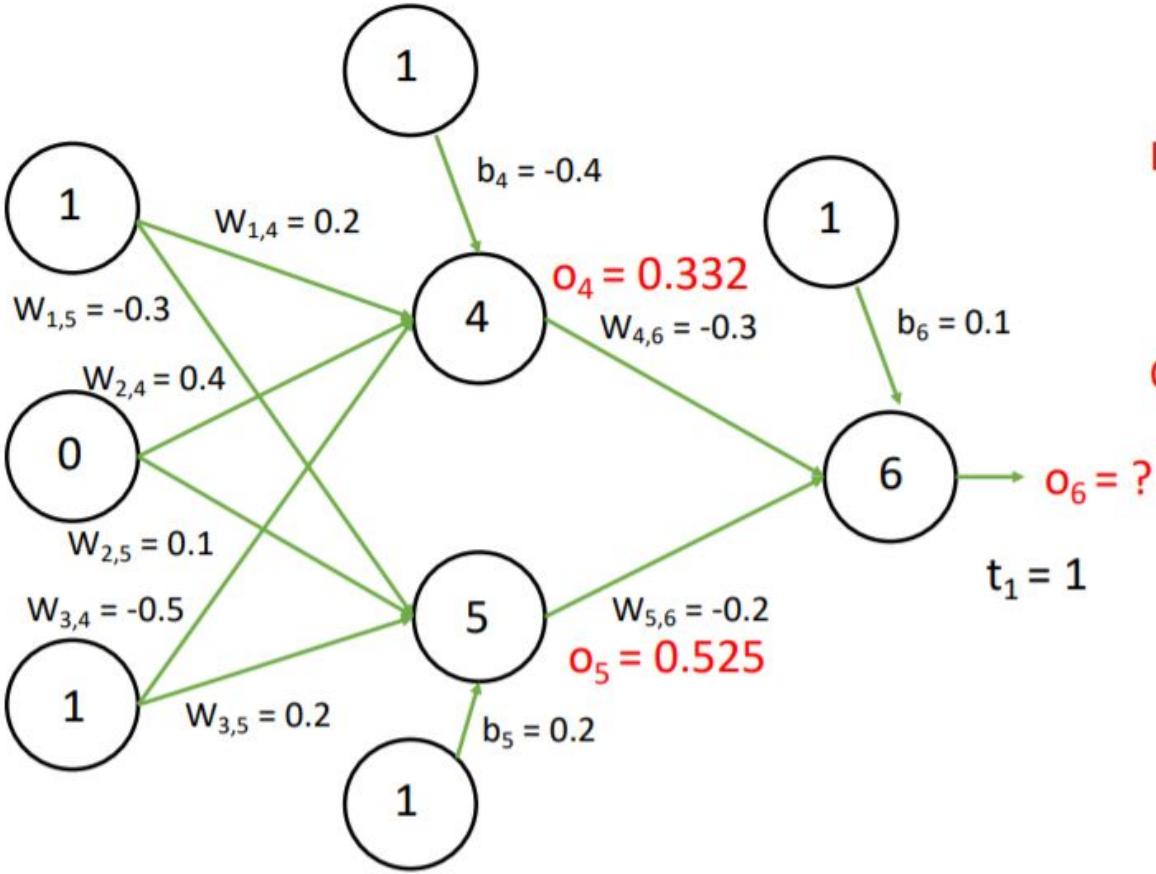
Output of node 5 (sigmoid function):

$o_5 = sigmoid(0.1)$

$o_5 = 1/(1+e^{-0.1})$

$o_5 = 0.525$

Backpropagation Example – Forward Pass



Input to node 6:

$i_6 = (0.332 \times -0.3 + 0.1 \times -0.2) + 0.1$

$i_6 = -0.105$

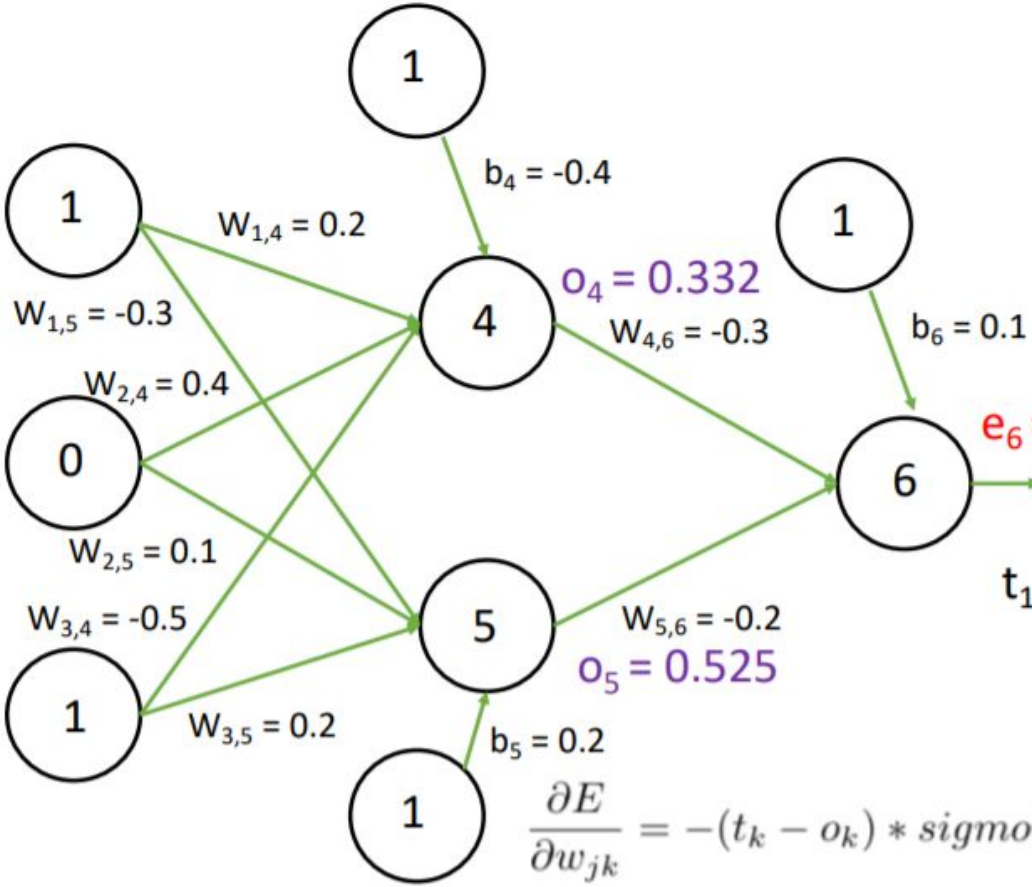
Output of node 6 (sigmoid function):

$o_6 = sigmoid(-0.105)$

$o_6 = 1/(1+e^{-(-0.105)})$

$o_6 = 0.474$

Backpropagation Example – Backward Pass



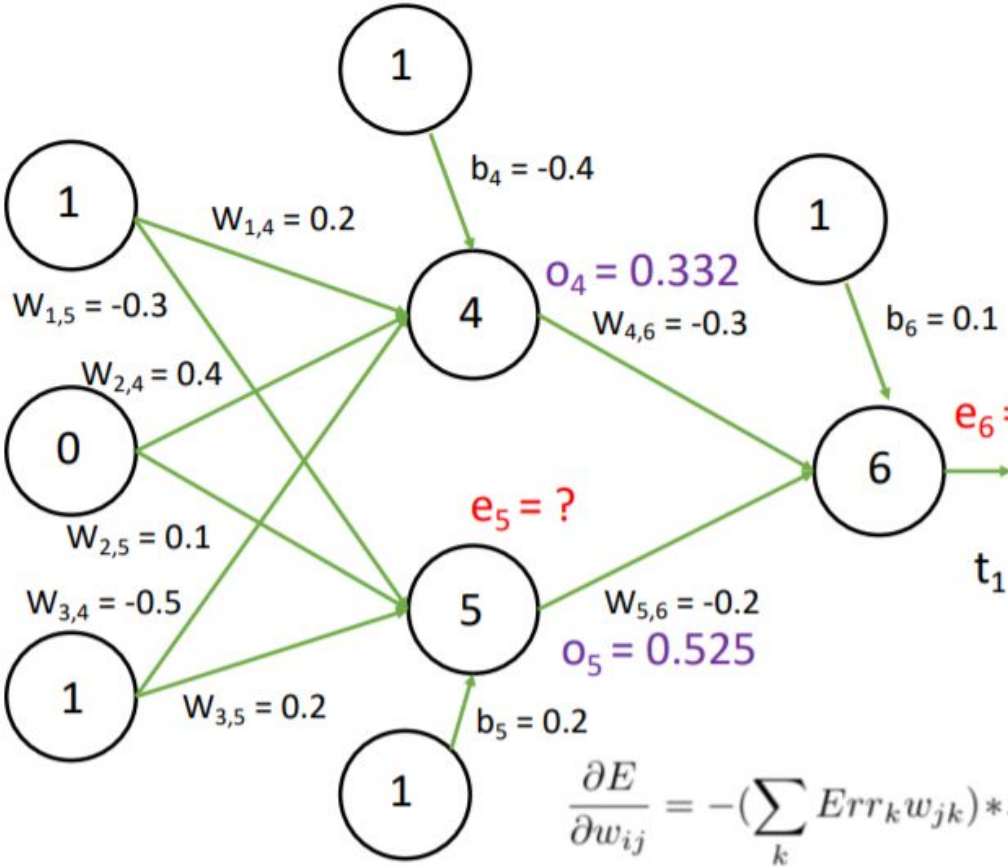
Error at node 6:

$e_6 = (1-0.474)\ (0.474)(1-0.474)$

$e_6 = 0.1311$

$$\frac{\partial E}{\partial w_{jk}} = -(t_k - o_k) * sigmoid(\sum_j w_{jk} * o_j) * (1 - sigmoid(\sum_j w_{jk} * o_j))$$

Backpropagation Example – Backward Pass



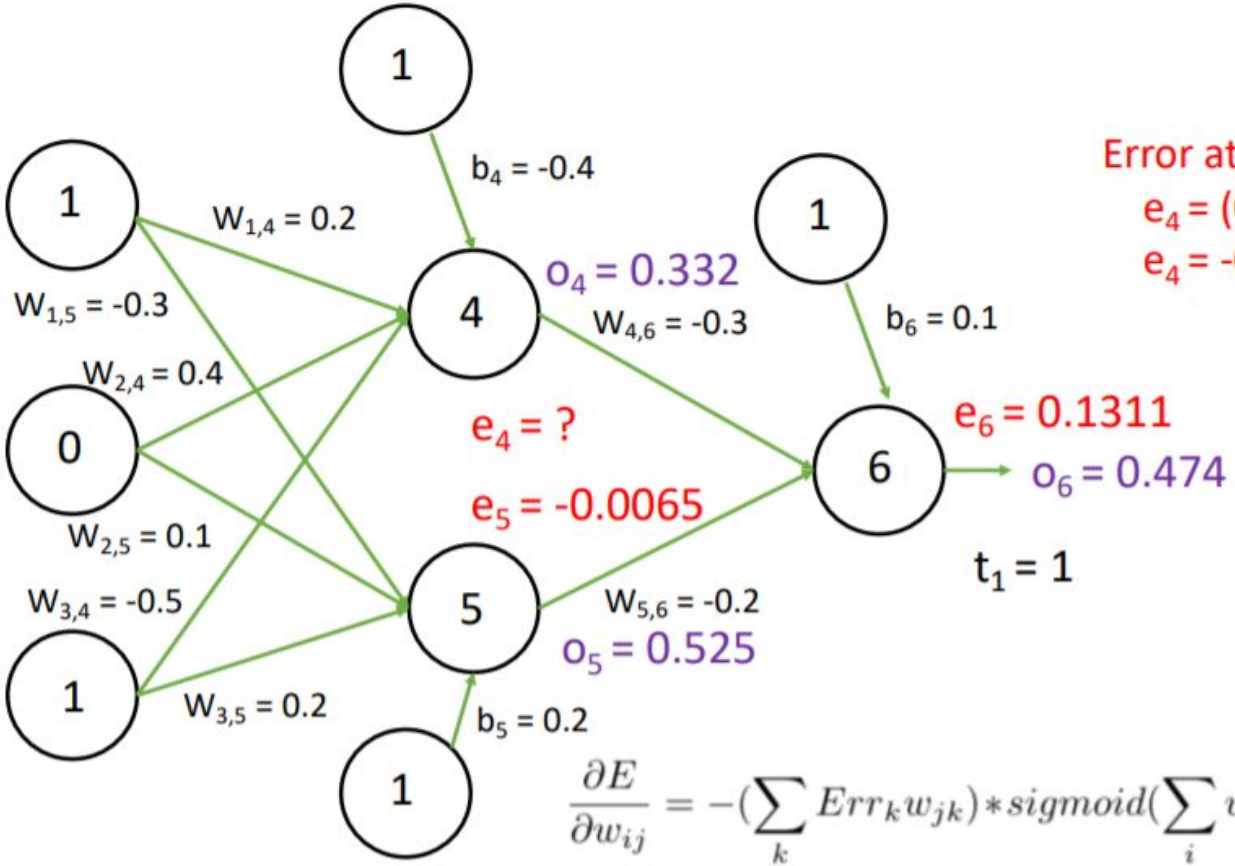
Error at node 5:

$e_5 = (0.1311)(-0.2)(0.525)(1-0.525)$

$e_5 = -0.0065$

$$\frac{\partial E}{\partial w_{ij}} = -(\sum_k Err_k w_{jk}) * sigmoid(\sum_i w_{ij} * o_i) * (1 - sigmoid(\sum_i w_{ij} * o_i))$$

Backpropagation Example – Backward Pass



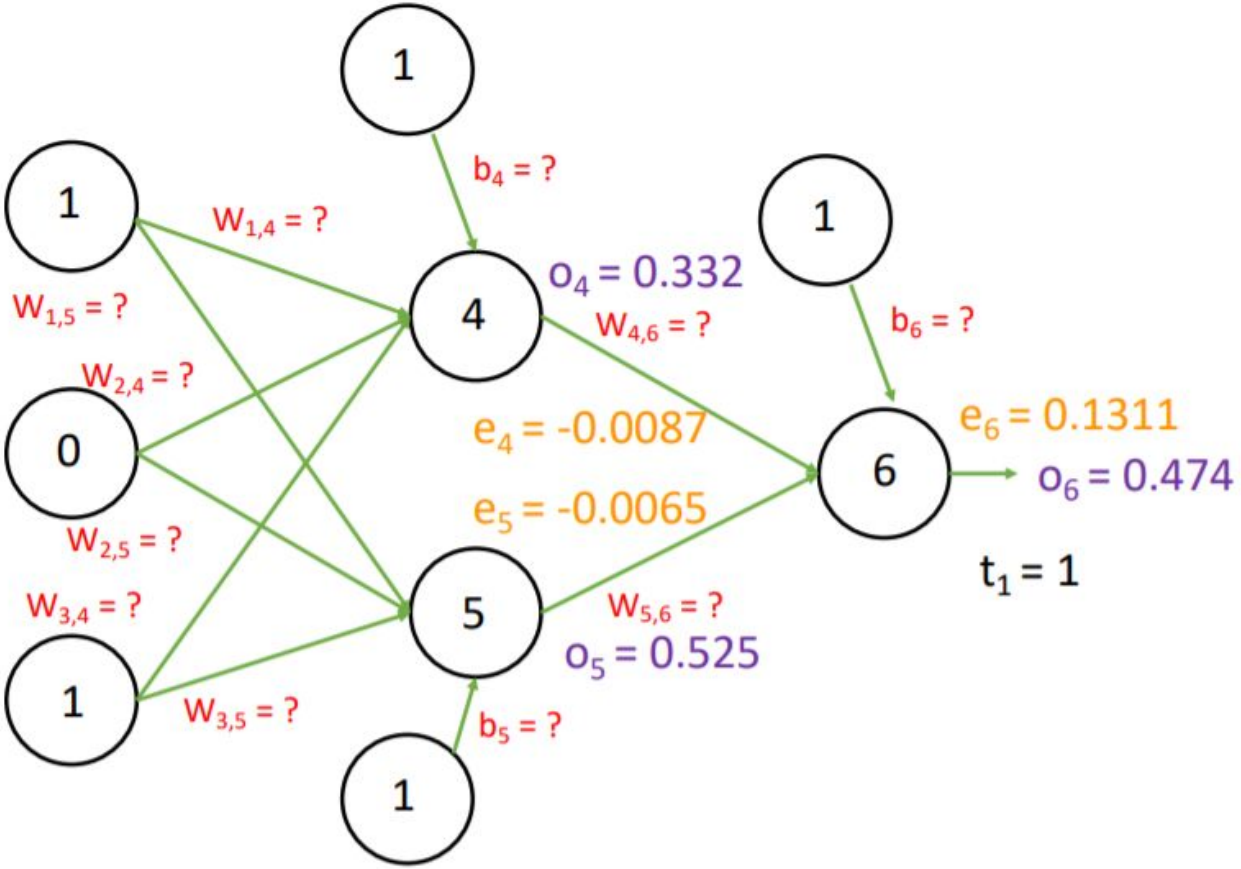
Error at node 4:

$e_4 = (0.1311)(-0.3)(0.332)(1-0.332)$

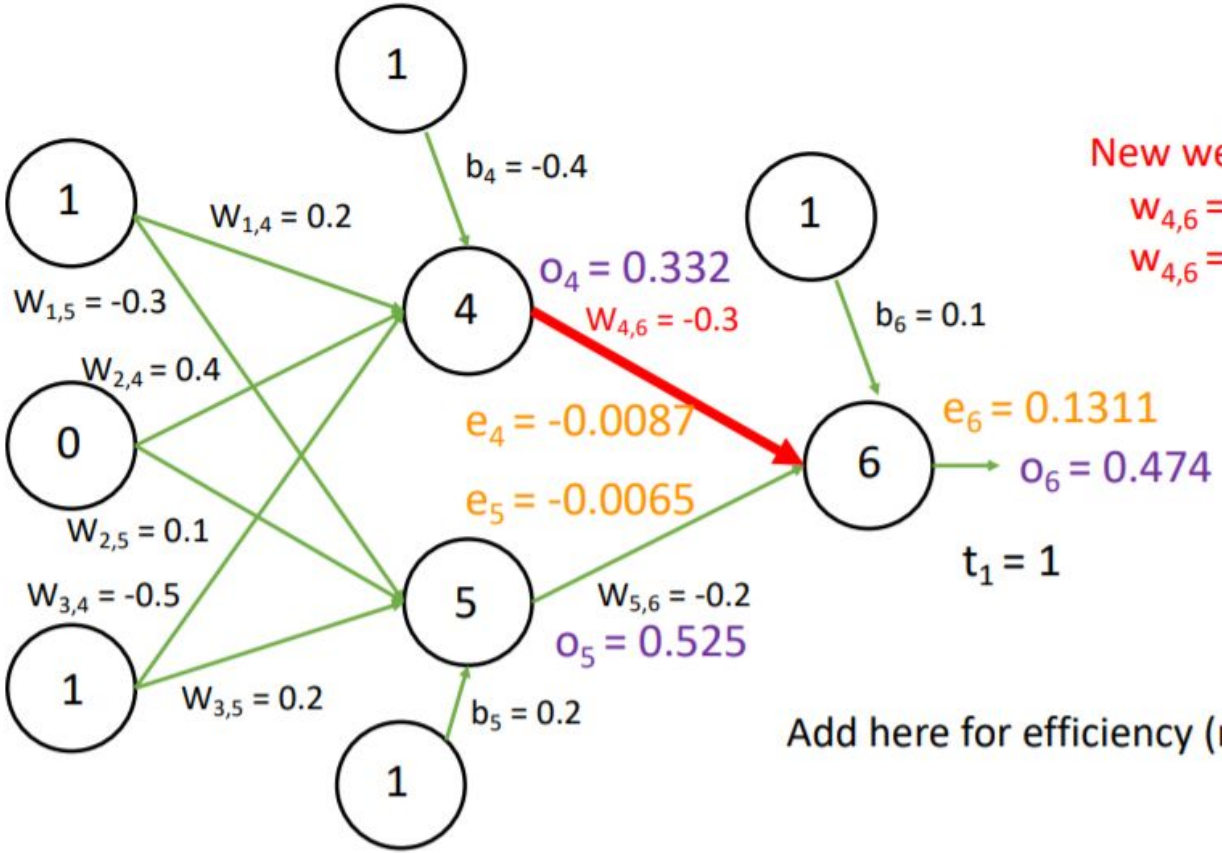
$e_4 = -0.0087$

$$\frac{\partial E}{\partial w_{ij}} = -(\sum_k Err_k w_{jk}) * sigmoid(\sum_i w_{ij} * o_i) * (1 - sigmoid(\sum_i w_{ij} * o_i))$$

Backpropagation Example – Update Weights



Backpropagation Example – Update Weights



New weights (learning rate = 0.9):

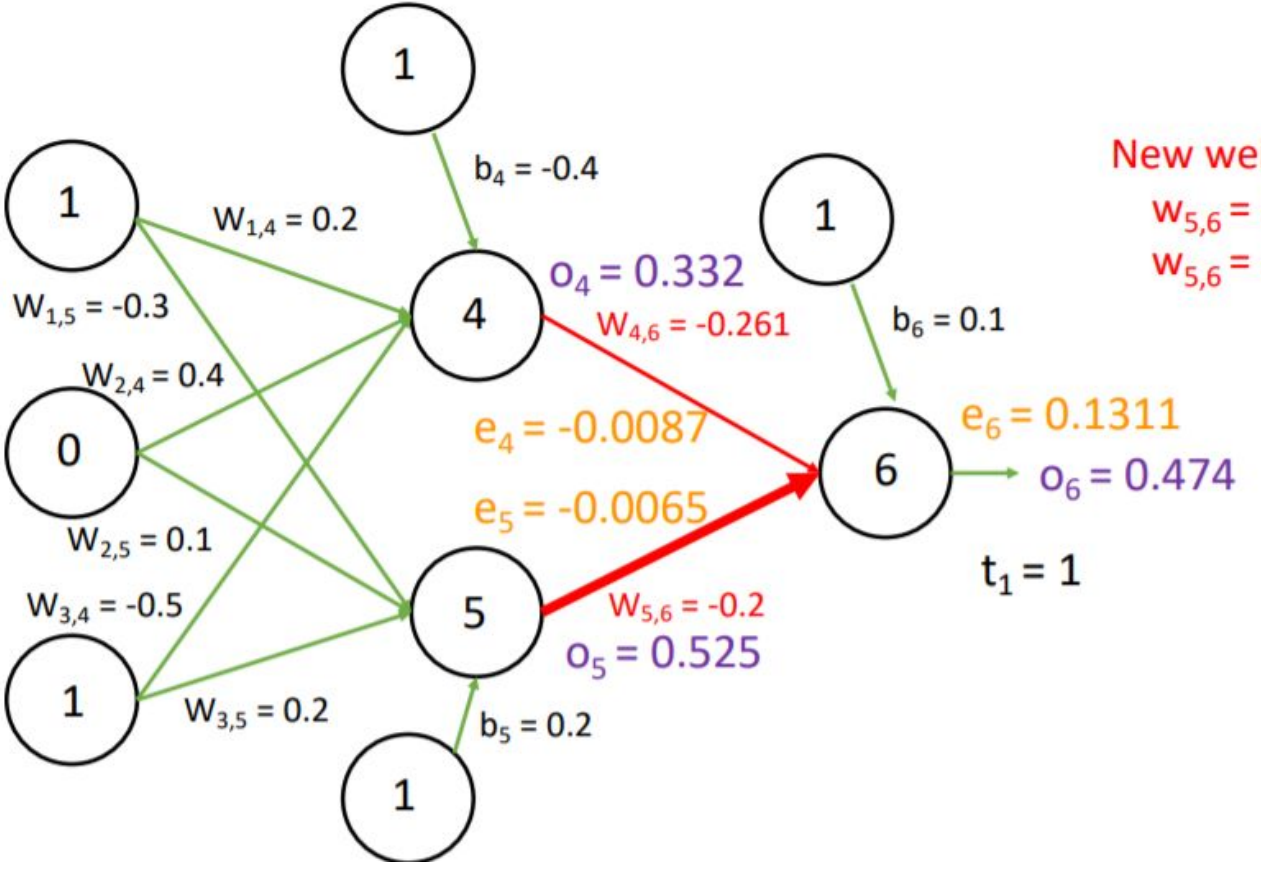
$w_{4,6} = -0.3 + (0.9)(0.1311)(0.332)$

$w_{4,6} = -0.261$

Add here for efficiency (removed from earlier equation)

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



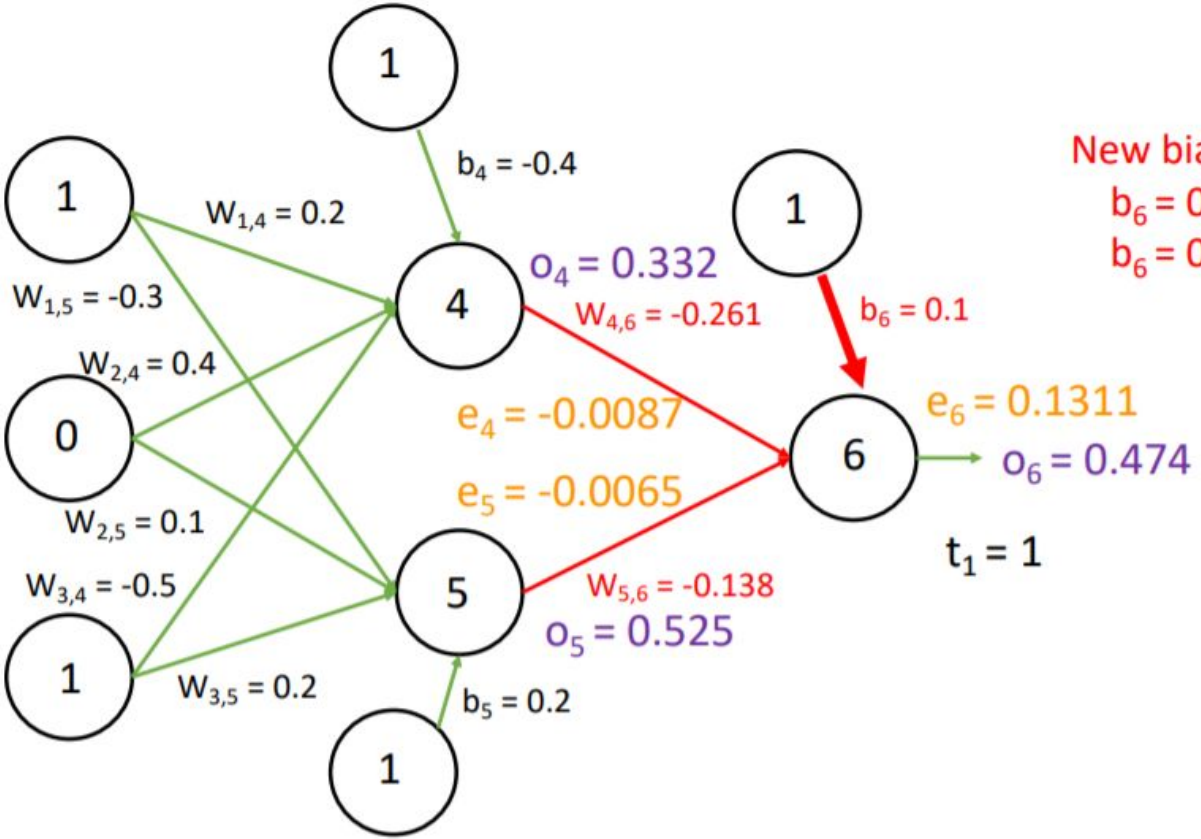
New weights (learning rate = 0.9):

$w_{5,6} = -0.2 + (0.9)(0.1311)(0.525)$

$w_{5,6} = -0.138$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



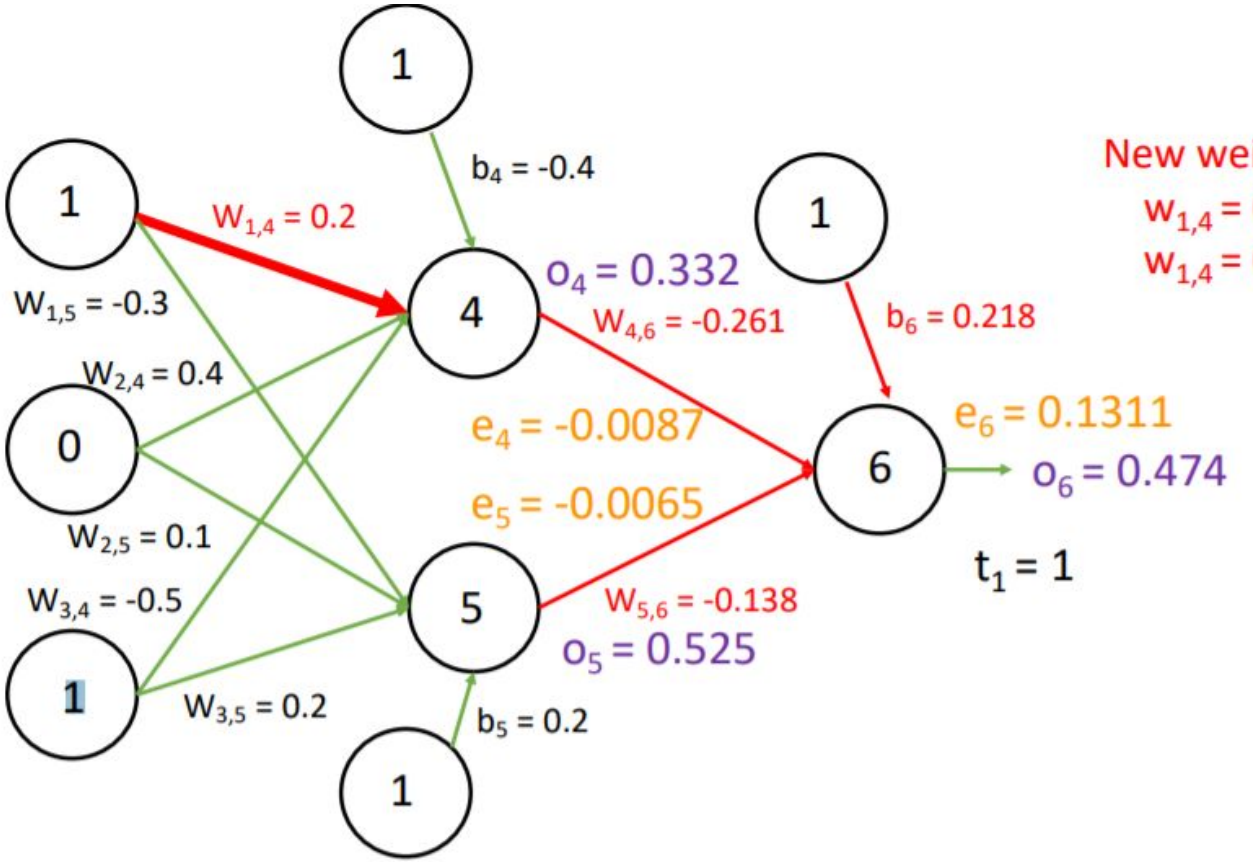
New bias (learning rate = 0.9):

$b_6 = 0.1 + (0.9)(0.1311)$

$b_6 = 0.218$

$$b_k = b_k + \eta Err_k$$

Backpropagation Example – Update Weights



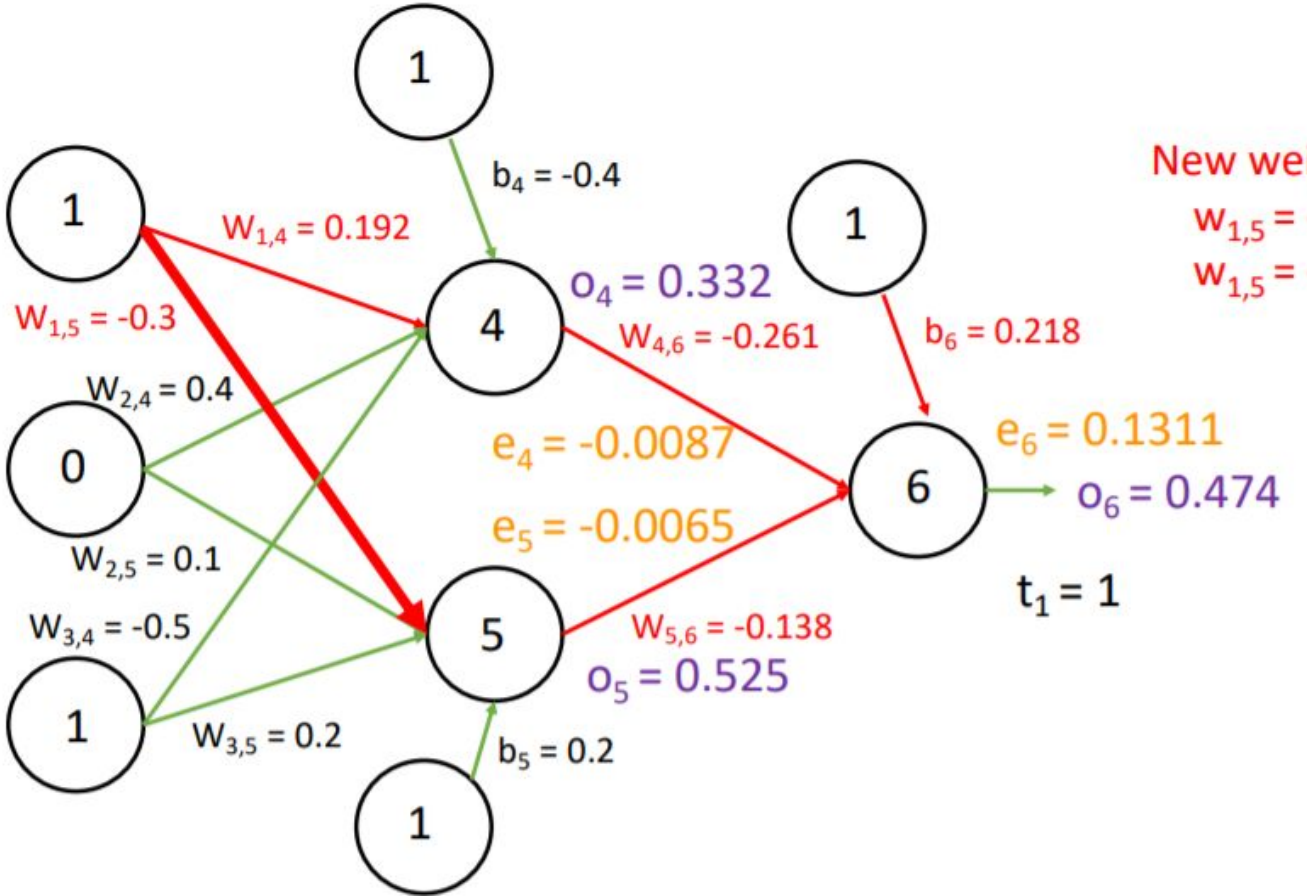
New weights (learning rate = 0.9):

$w_{1,4} = 0.2 + (0.9)(-0.0087)(1)$

$w_{1,4} = 0.192$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



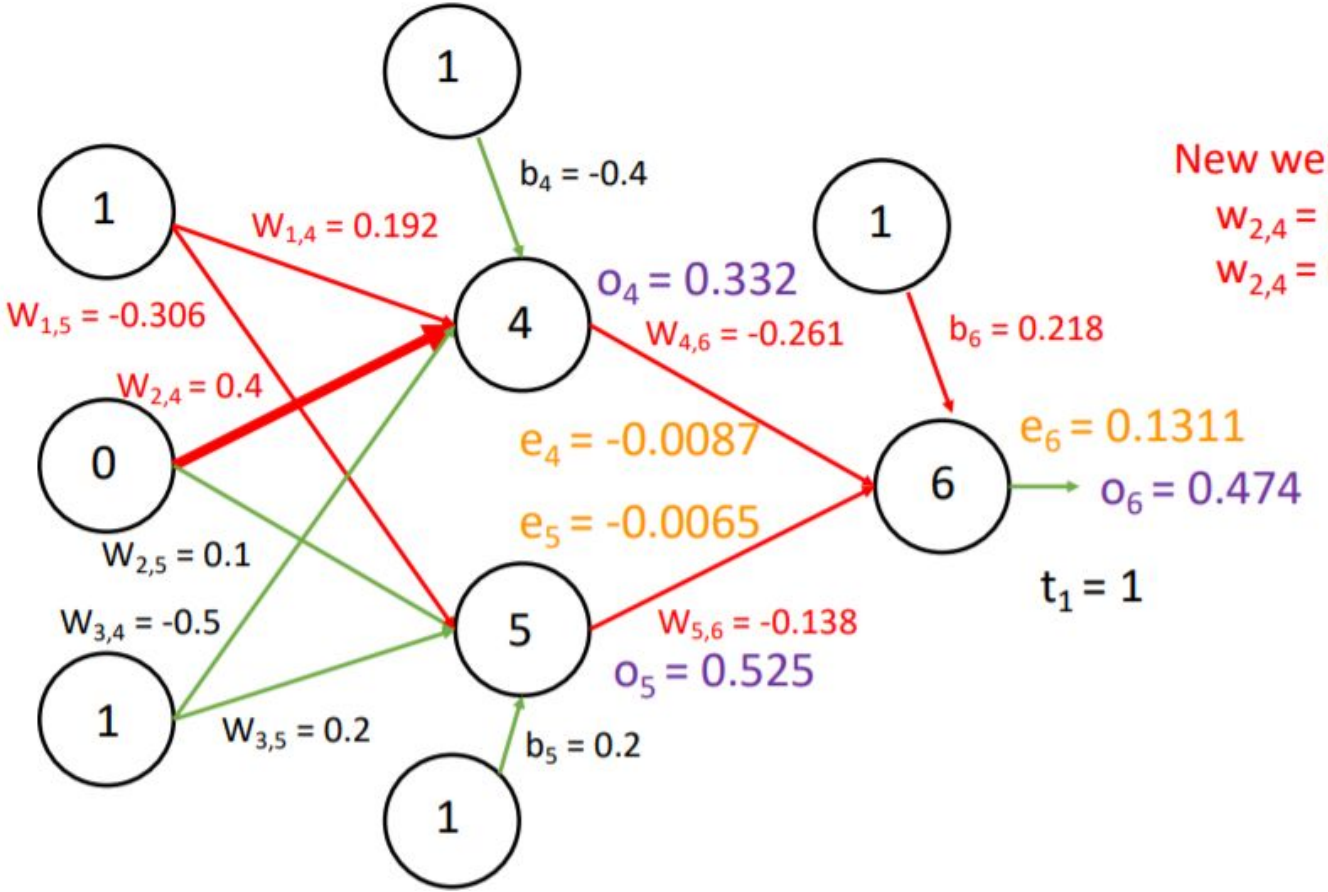
New weights (learning rate = 0.9):

$w_{1,5} = -0.3 + (0.9)(-0.0065)(1)$

$w_{1,5} = -0.306$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



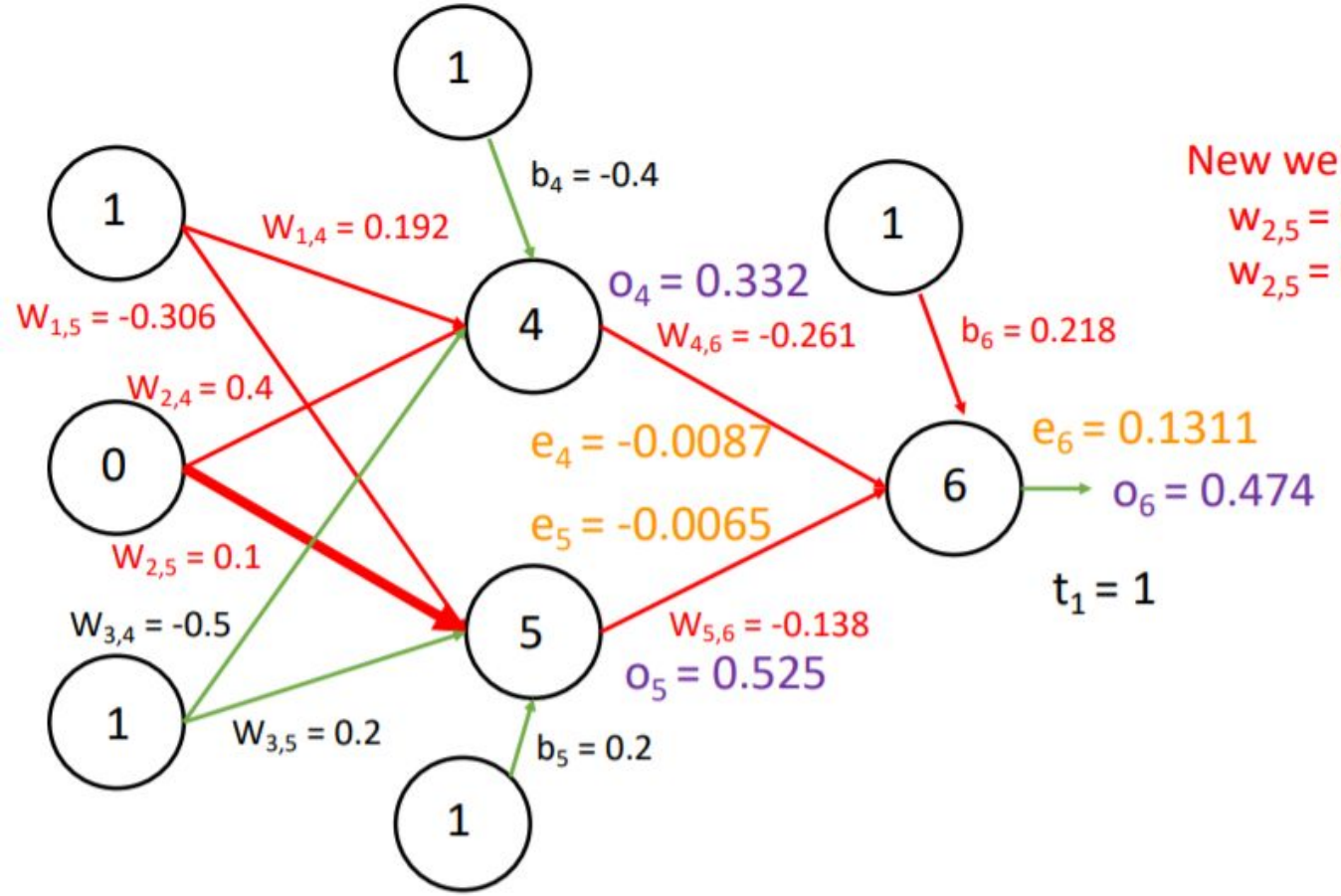
New weights (learning rate = 0.9):

$w_{2,4} = 0.4 + (0.9)(-0.0087)(0)$

$w_{2,4} = 0.4$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



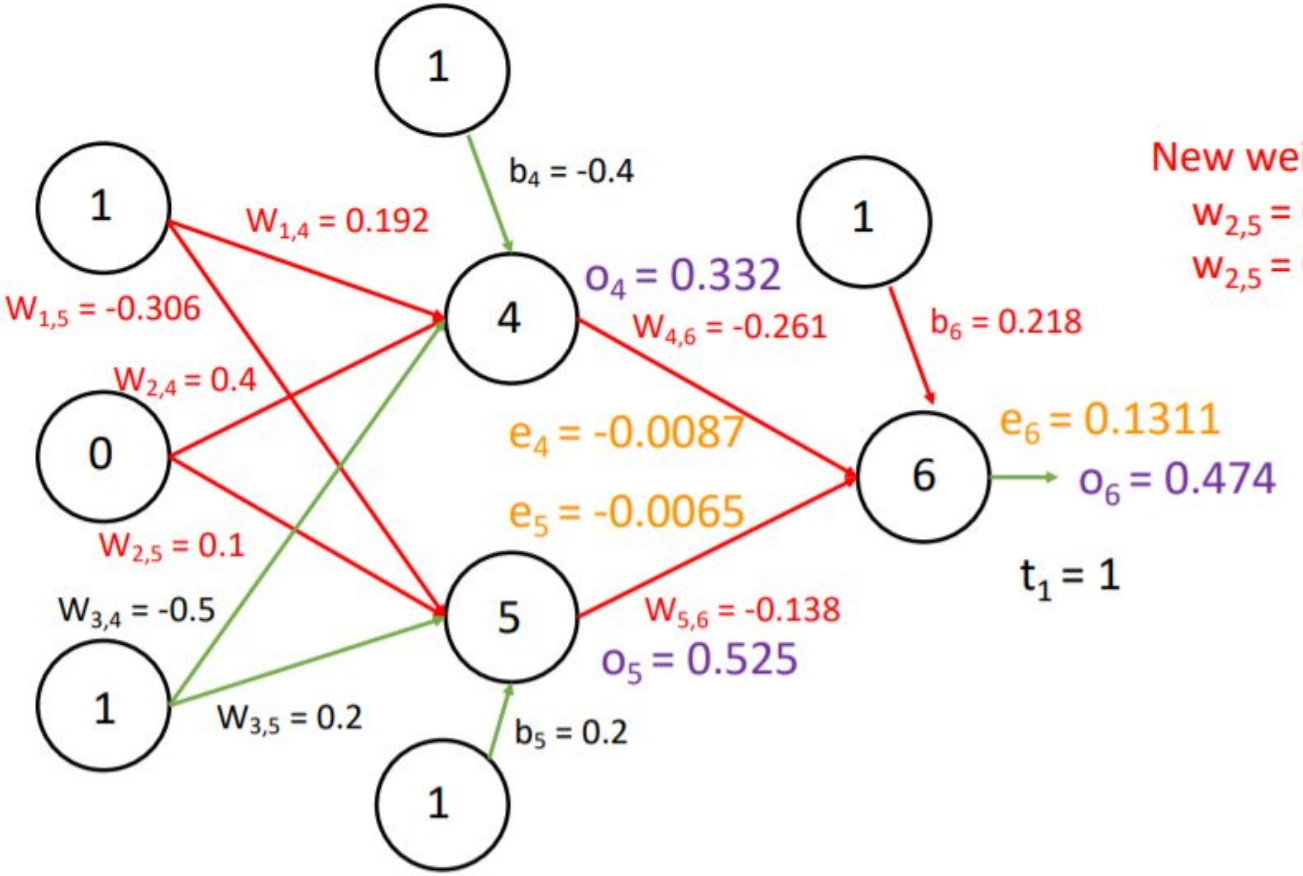
New weights (learning rate = 0.9):

$w_{2,5} = 0.1 + (0.9)(-0.0065)(0)$

$w_{2,5} = 0.1$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



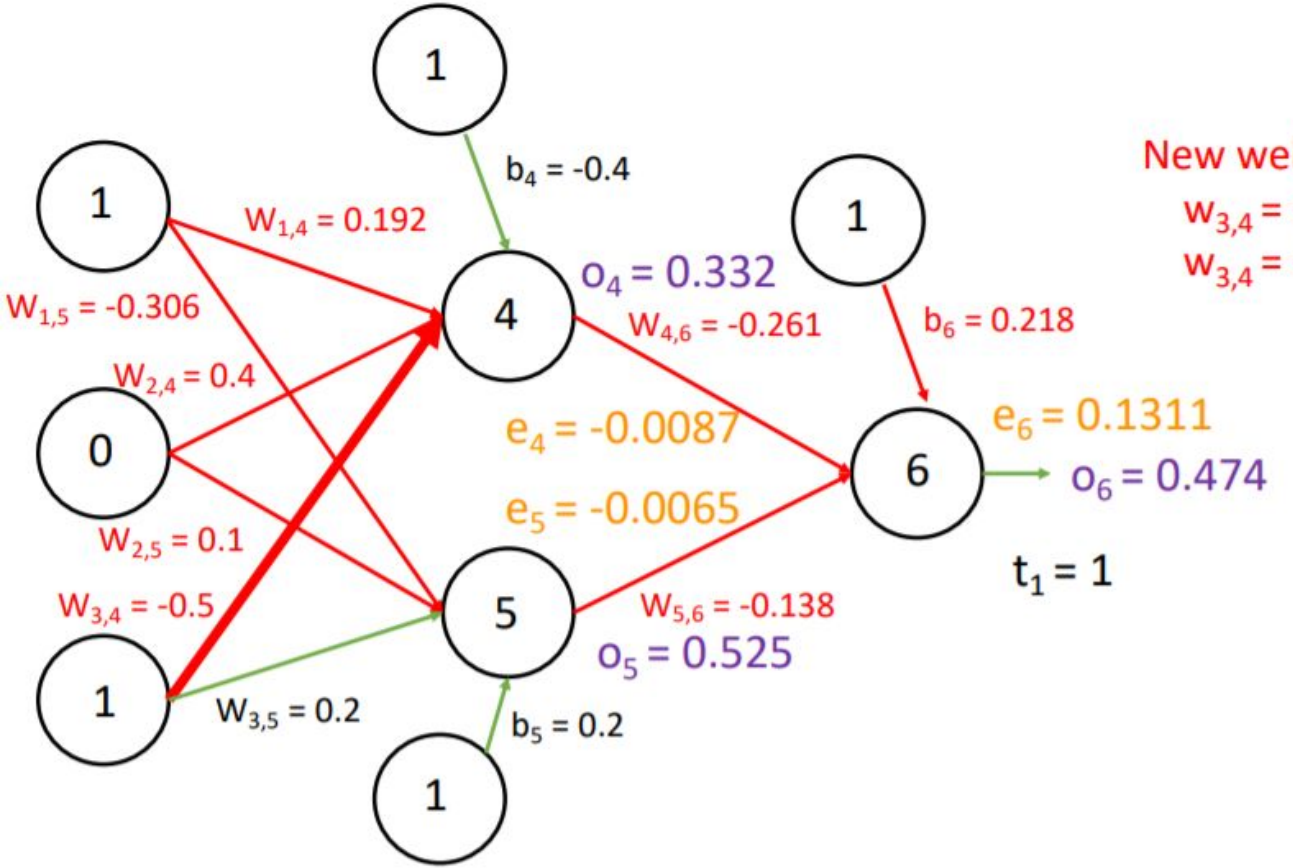
New weights (learning rate = 0.9):

$w_{2,5} = 0.1 + (0.9)(-0.0065)(0)$

$w_{2,5} = 0.1$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



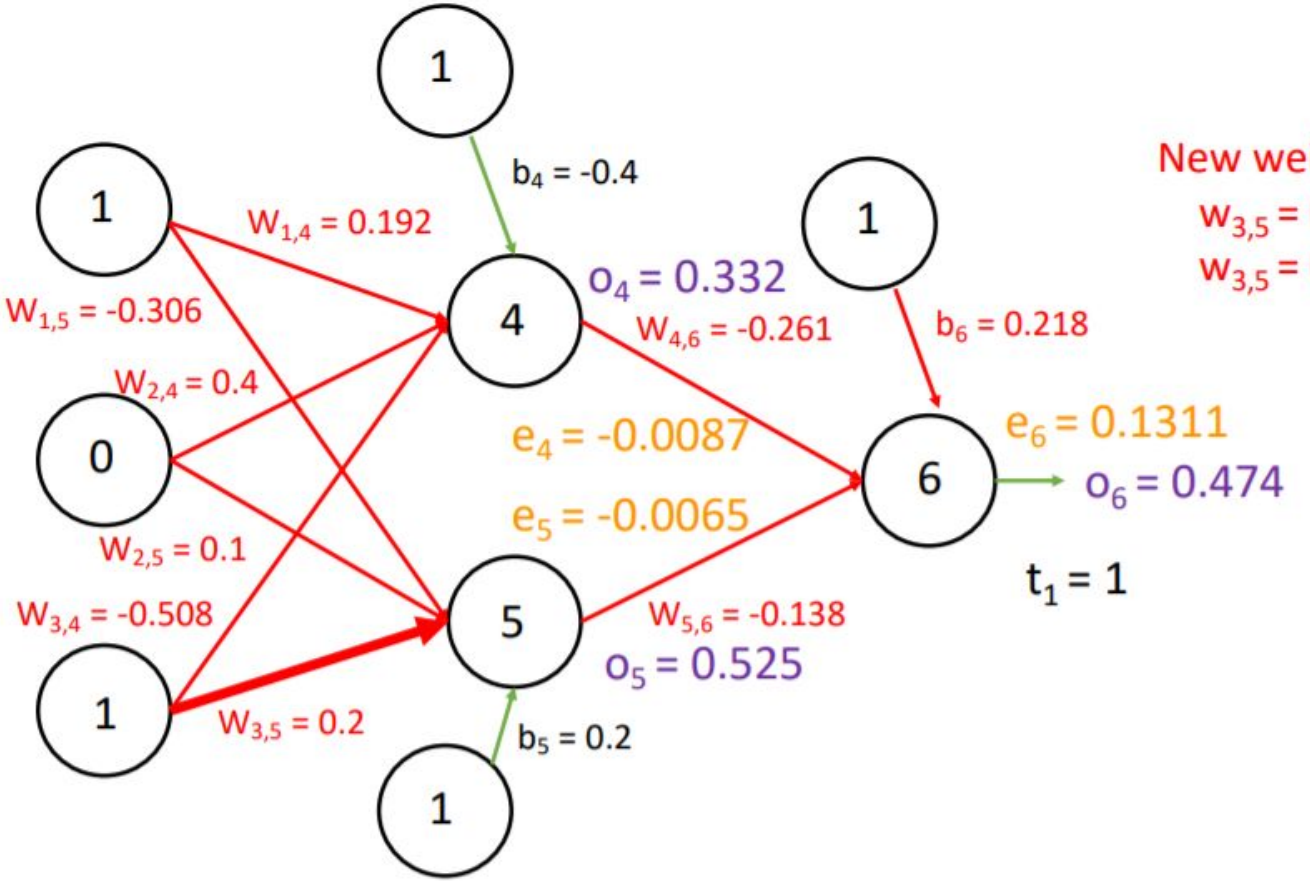
New weights (learning rate = 0.9):

$w_{3,4} = -0.5 + (0.9)(-0.0087)(1)$

$w_{3,4} = -0.508$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



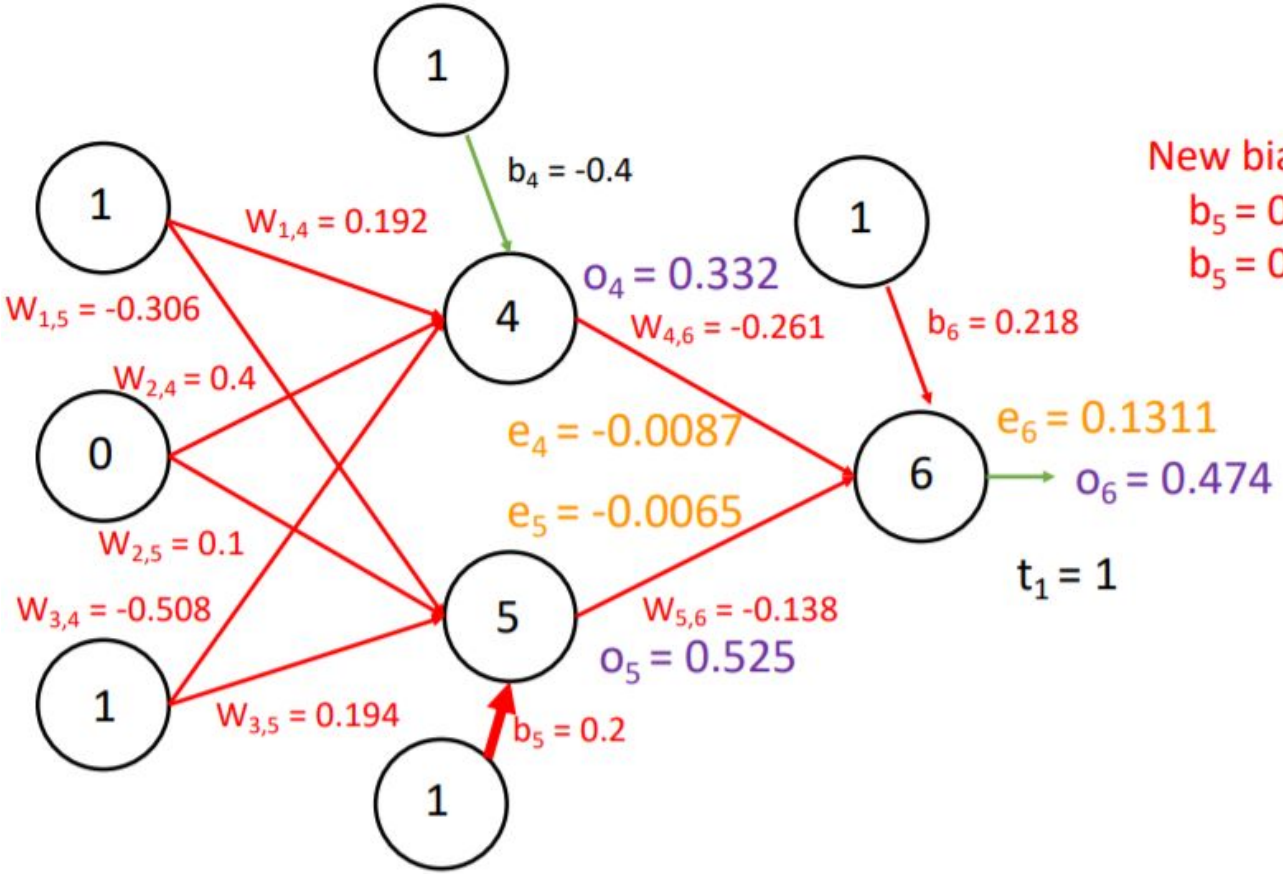
New weights (learning rate = 0.9):

$w_{3,5} = 0.2 + (0.9)(-0.0065)(1)$

$w_{3,5} = 01.94$

$$w_{jk} = w_{jk} + \eta Err_k o_j$$

Backpropagation Example – Update Weights



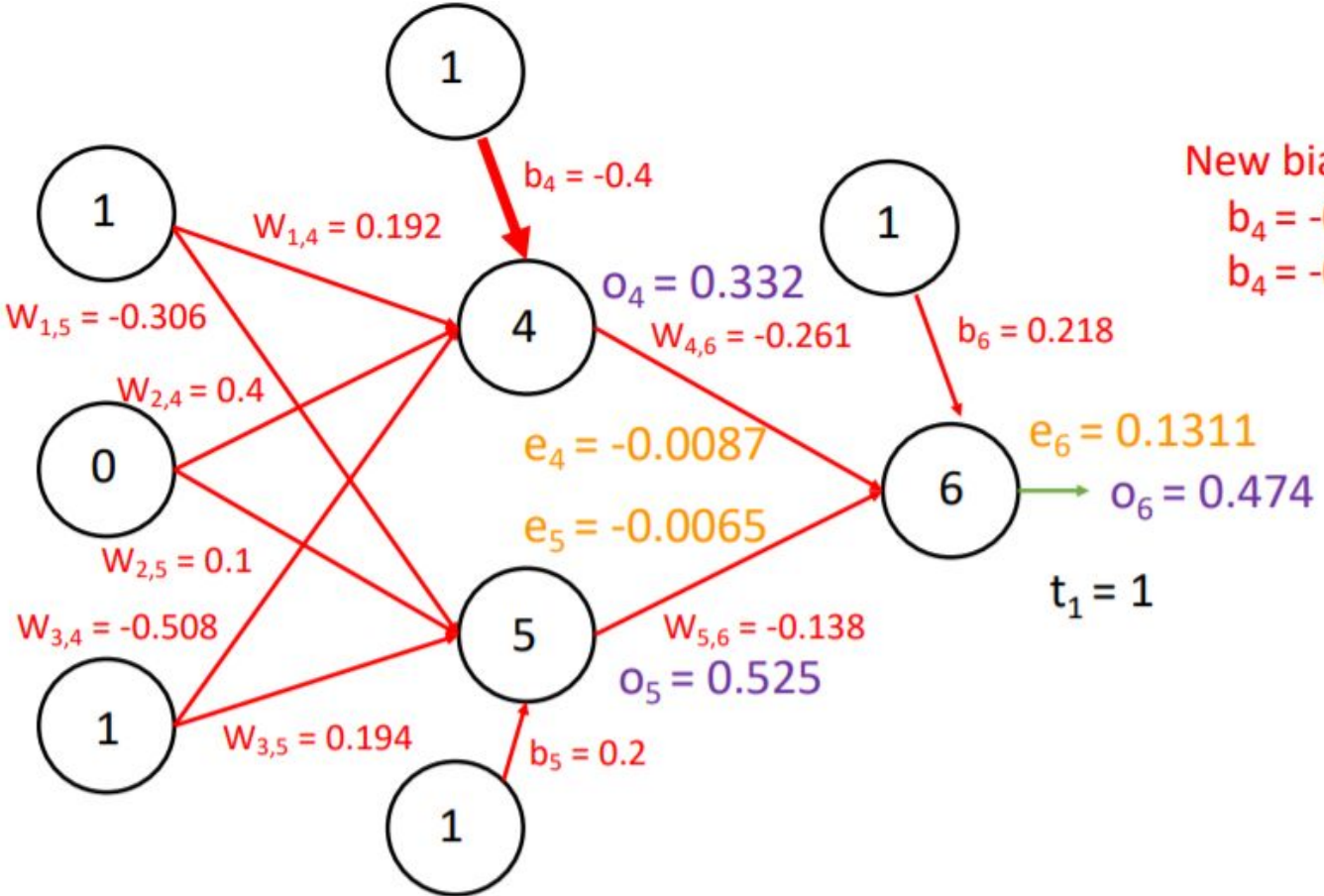
New bias (learning rate = 0.9):

$b_5 = 0.2 + (0.9)(-0.0065)$

$b_5 = 0.194$

$$b_k = b_k + \eta Err_k$$

Backpropagation Example – Update Weights



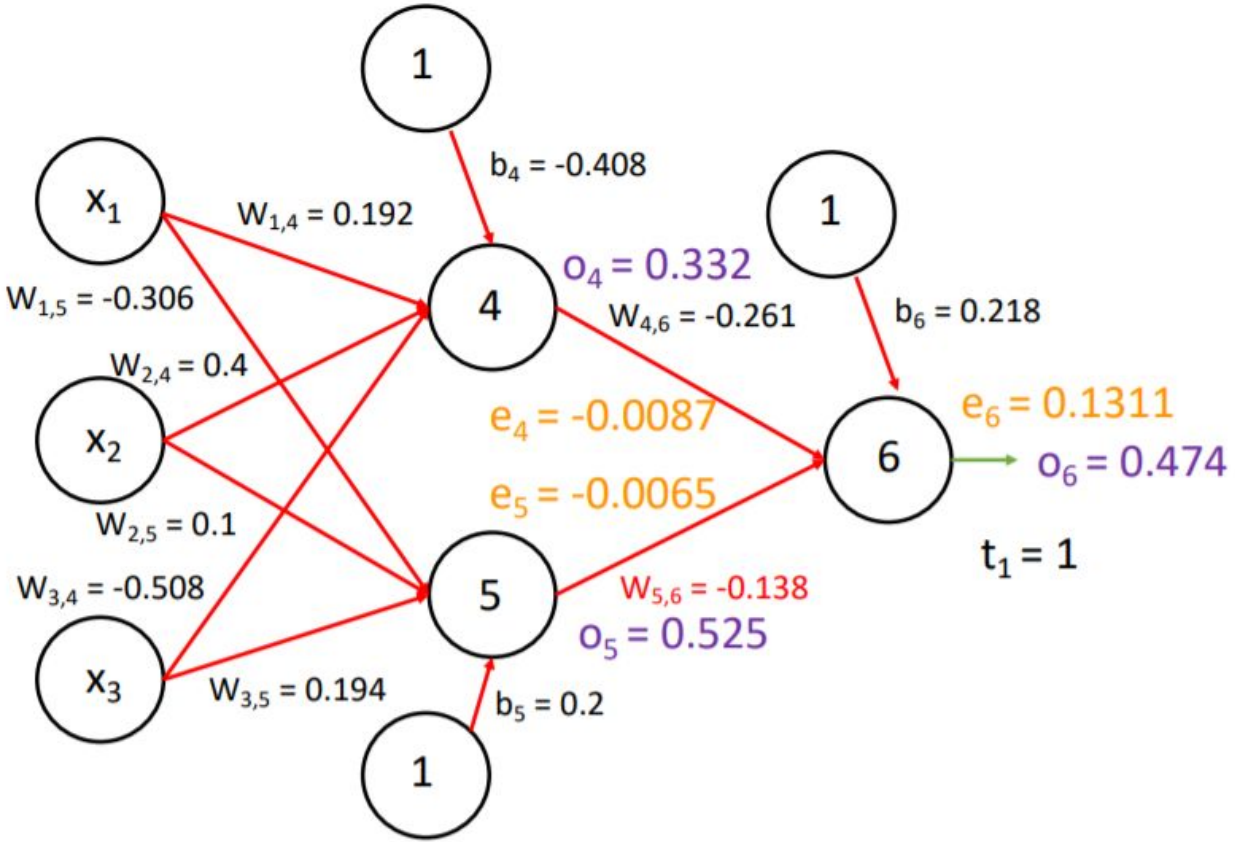
New bias (learning rate = 0.9):

$b_4 = -0.4 + (0.9)(-0.0087)$

$b_4 = -0.408$

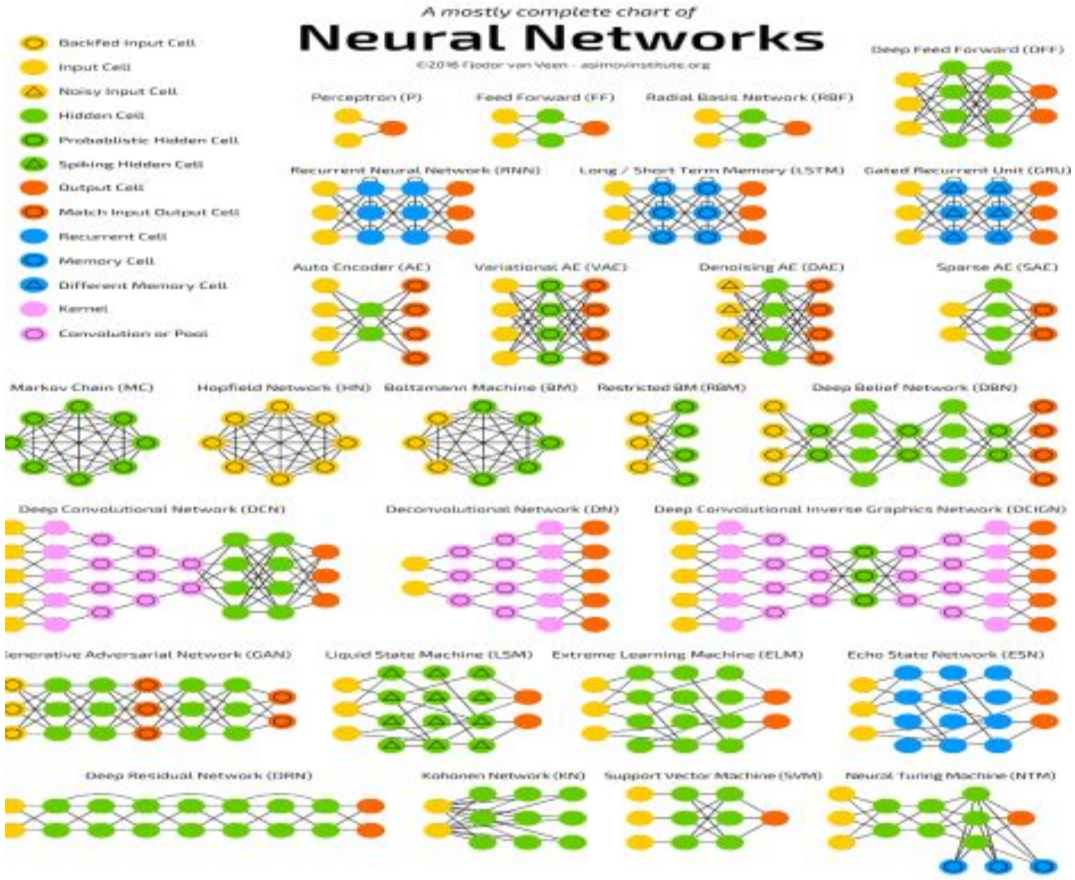
$$b_k = b_k + \eta Err_k$$

One iteration complete!



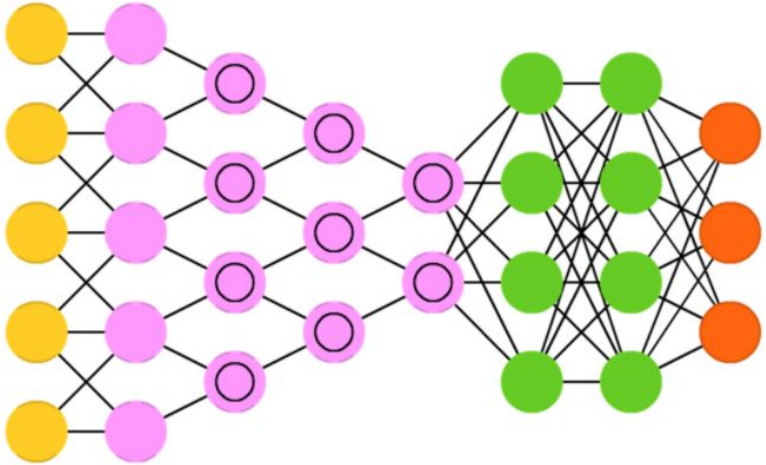
Some Notable Neural Network Architectures

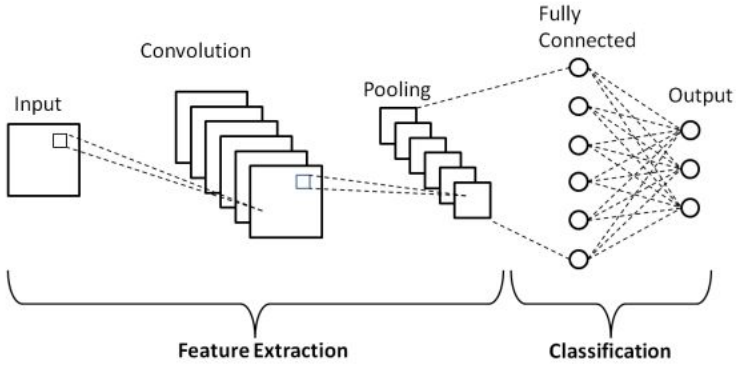
- Feedforward Neural Network (FFNN or DNN), as seen so far in examples. Others include:
- Recurrent Neural Networks (RNN)
- Long/Short Term Memory (LSTM)
- Autoencoders (AE)
- Convolutional Neural Networks (CNN)
- Generative Adversarial Networks (GAN)
- …and many, many more!



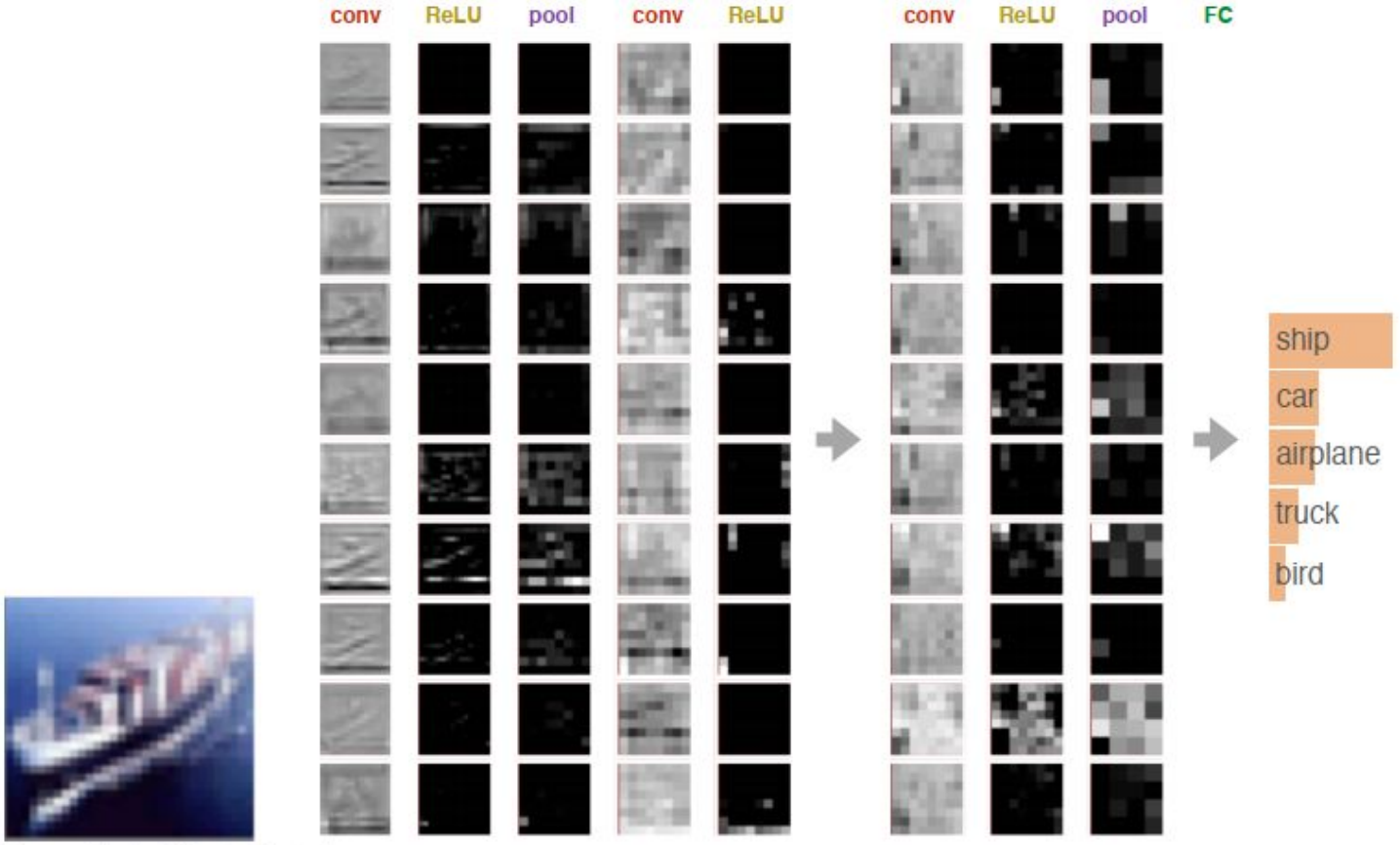
Convolutional Neural Networks (CNN)

- Primarily used in image classification
- Not a fully connected network – each node is only concerned with neighboring cells in convolutional layers
- Convolutional layers perform convolution between the weights and the scanned pixels
- Conv layers are followed by pooling layers (typically max pooling), which takes small regions of the conv output and takes the max value present
- Typically terminates with a FFNN to the end





Convolutional Neural Networks (CNN)



Source: http://cs231n.stanford.edu

2D Convolution and Maxpooling

	Input				Kernel			Output	
	1	3	2	1	1	2	3		
	1	3	3	1	0	1	0	23	22
	2	1	1	3	2	1	2	31	26
	3	2	3	3					

	Input						Kernel			Output			
	0	0	0	0	0	0							
	0	1	3	2	1	0	1	2	3	8	14	13	8
	0	1	3	3	1	0	0	1	0	16	23	22	10
	0	2	1	1	3	0	2	1	2	20	31	26	17
	0	3	2	3	3	0				10	9	15	10
	0	0	0	0	0	0							

Single depth slice

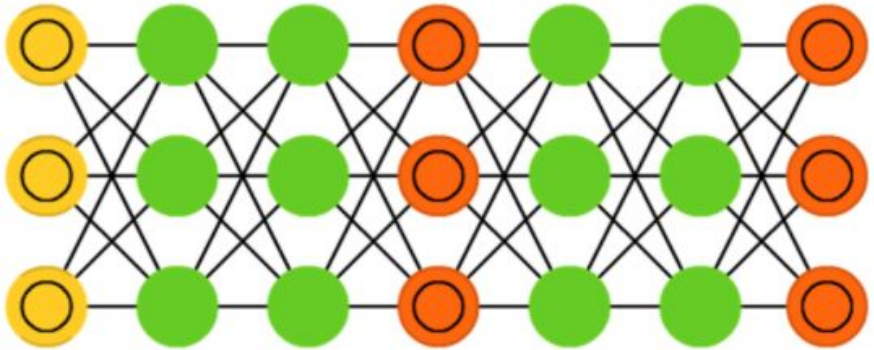
1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

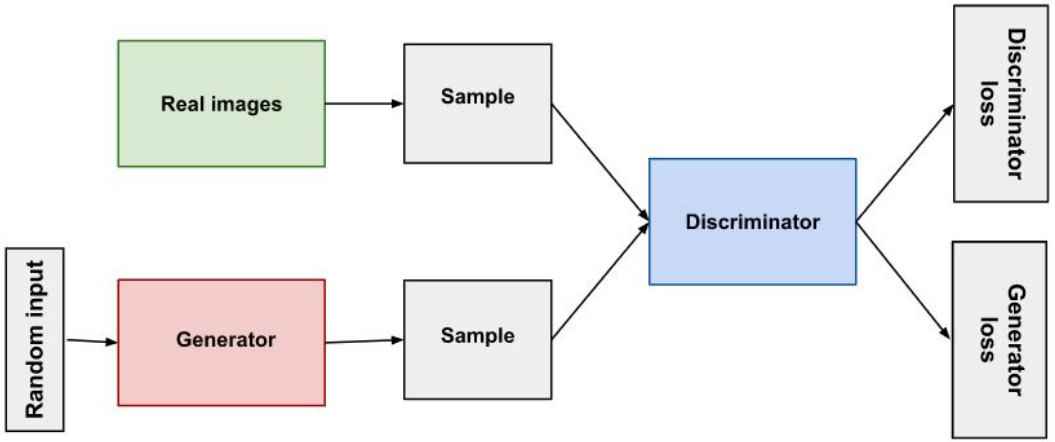
max pool with 2x2 filters and stride 2

6	8
3	4

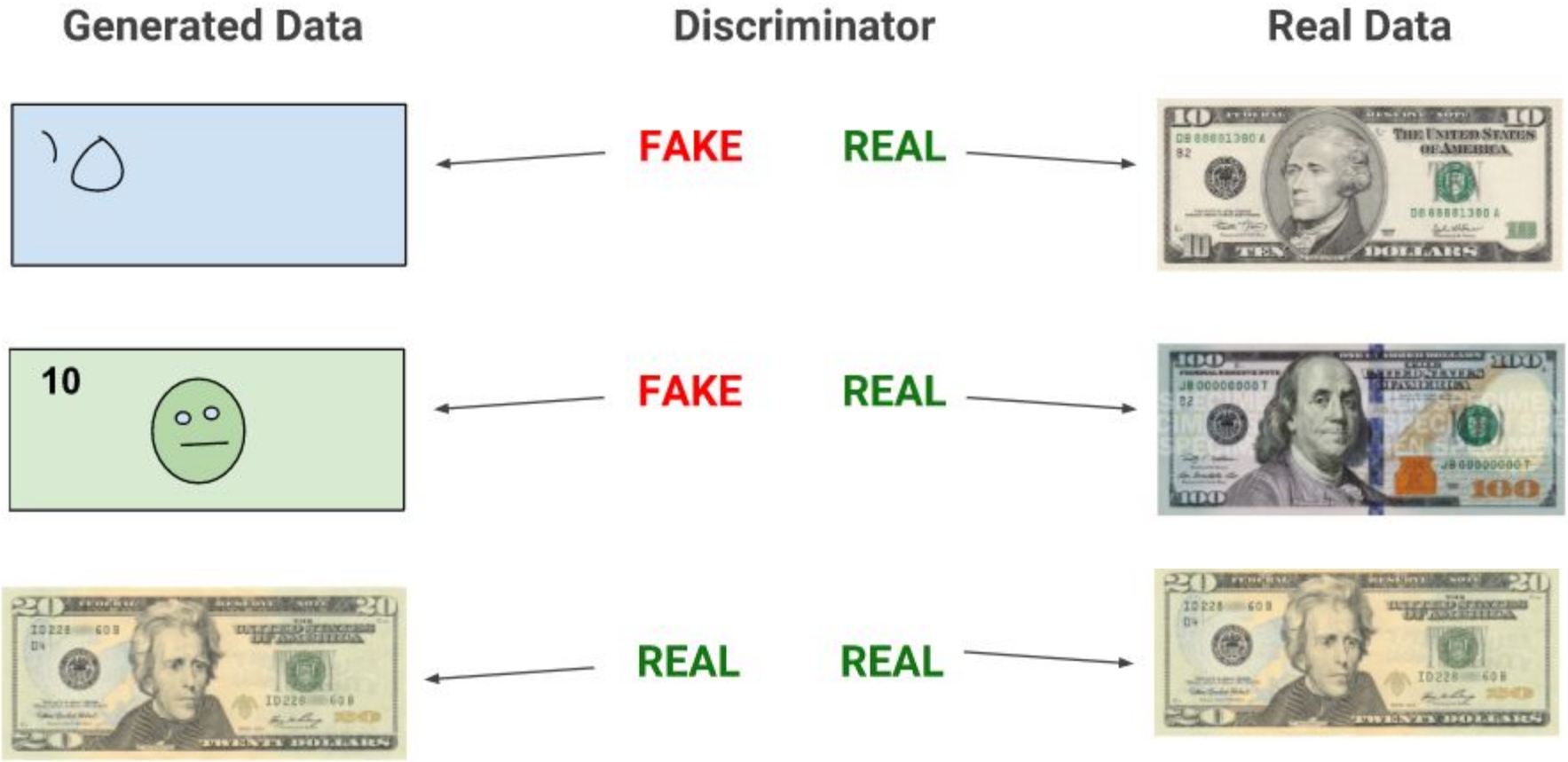
Generative Adversarial Networks (GAN)

- Not one, but two networks (usually a combination of FFNN and CNN)
- Generative network makes content, and discriminating network judges content
- These two will compete against each other
- Pretty difficult to train due to competing dynamics of the two networks
- Generator does not train when discriminator trains, and vice versa



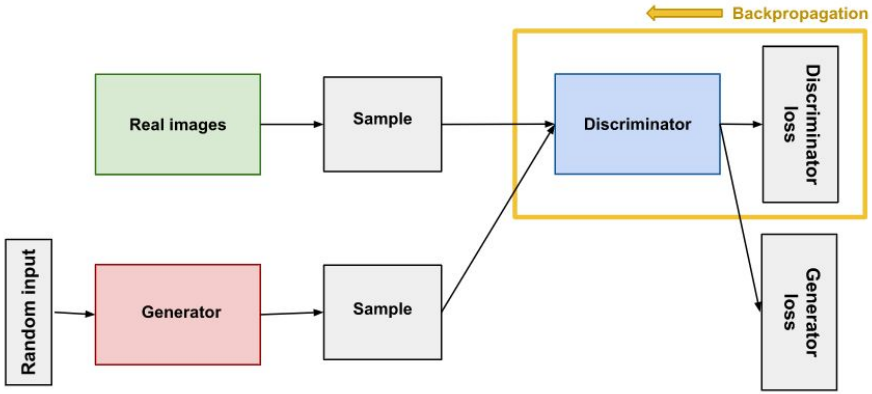


Generative Adversarial Networks



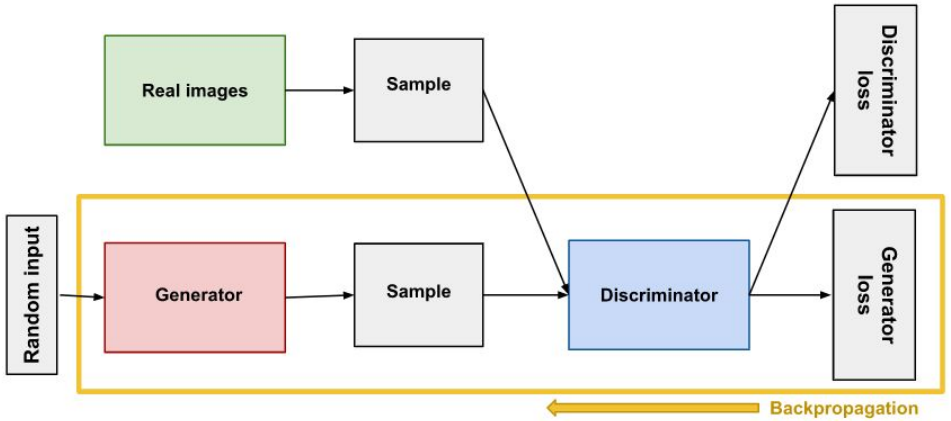
Discriminator (GAN)

- Training data consists of real data (such as real pictures of people, positive class) and fake data created by the generator(negative class)
- Generator weights stay constant while it produces examples for the discriminator training
- During training, the discriminator:
  1. Classifies both real and fake data from the generator
  2. The discriminator loss penalizes the discriminator for misclassification
  3. Discriminator weights are updated via backpropagation from the discriminator loss through the discriminator network



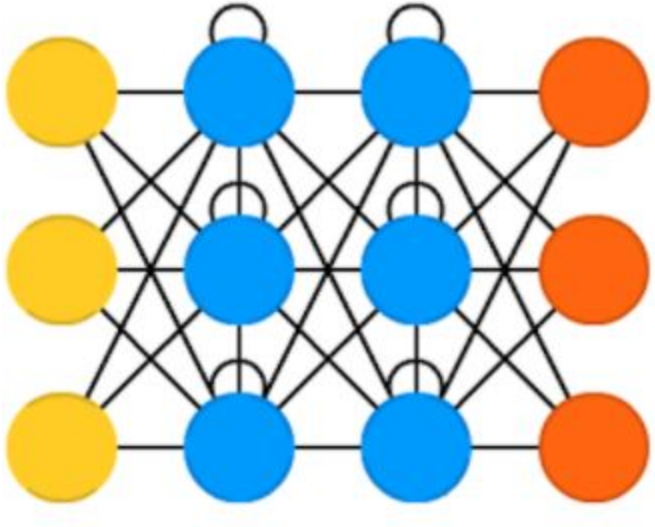
Generator (GAN)

- Takes random noise as input, and produces meaningful output to fool the discriminator
- Generator loss is produced at the discriminator □ backpropagation starts at the output and flows back through the discriminator into the generator
- Only the generator weights are updated from the gradients obtained from backpropagation □ discriminator stays constant during generator training



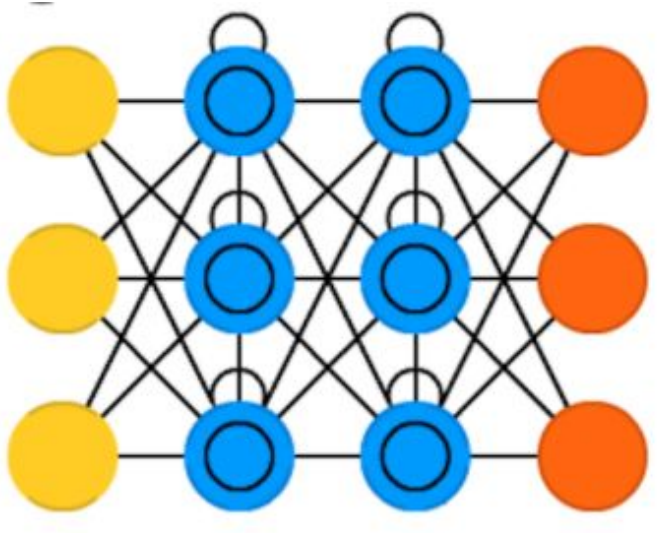
Recurrent Neural Networks

- Time twist: these are not stateless!
- Neurons are fed information from the previous pass, as well as the previous layer
- The order in which you feed the input and train the network matters as a result
- Decent choice for advancing or completing information (e.g. your phone's autocomplete)
- Suffers from vanishing/exploding gradients issues



Long/Short Term Memory (LSTM)

- Instead of using recurrent cells (that have feedback from past inputs), LSTM uses explicitly defined memory cells to counter the vanishing/exploding gradient issue
- Each neuron has a memory cell and three gates: input, output, and forget
- Has been shown to learn complex sequences (such as composing music)



Autoencoders (AE)

- Basic idea is to compress information via this architecture
- Hourglass shape – everything before the middle used for encoding, and after is used for decoding
- Can train with backpropagation and see if the output cells generate the input back